DeepSeek-V4.1-Flash：极致压缩 KV 缓存

Pushing the Limits of KV Cache Compression

DeepSeek-AI

中文原生版

DeepSeek-V4.1-Flash：极致压缩 KV 缓存

Pushing the Limits of KV Cache Compression

DeepSeek-AI

research@deepseek.com

摘　要

长程智能体的广泛使用，使模型负载的输入量持续走高。尽管已有工作大幅降低了长上下文计算的开销，预填充（prefill）依然代价高昂，庞大的 KV 缓存也持续压迫 HBM 与 SSD 容量以及数据传输带宽。计算、存储与带宽三方面的需求叠加，构成了进一步降低部署成本的首要瓶颈。为应对这一挑战，我们提出 **DeepSeek-V4.1-Flash**：一个多模态混合专家（MoE）模型，主干参数量 552B，支持最长一百万个词元的上下文。凭借因果编码器–解码器（CED）架构，模型在解码阶段每词元激活 16B 参数、在预填充阶段仅激活 8B 参数，显著提升了智能体负载的成本效率。为逼近 KV 缓存压缩的极限，DeepSeek-V4.1-Flash 把压缩稀疏注意力 2（CSA2）中的跨层 KV 缓存复用与 FP4 KV 缓存结合起来。这些设计把其全局 KV 缓存占用（始终驻留 HBM）降至每词元 890 字节，约为 DeepSeek-V4-Flash 对应占用的 1/4。此外，借助名为 SWA 有界重放的部署优化，DeepSeek-V4.1-Flash 把持久化 KV 缓存占用（始终位于 SSD 或主机内存）降至 DeepSeek-V4-Flash 的约 1/8。尽管 KV 缓存占用大幅缩小，模型的性能仍显著优于基线。我们还精简了 DeepSeek-V4 的架构，并引入若干高效的架构扩展。DeepSeek-V4.1-Flash 在包含 45T 词元的多模态语料上预训练，并经过充分的后训练，在多种纯文本与多模态智能体场景中均表现强劲。模型权重见 https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash。

关键词：混合专家；KV 缓存压缩；稀疏注意力；多模态；强化学习；智能体

目录

第 1 章 引言 **5**

第 2 章 架构 **9**
2.1 概述 9
2.2 因果编码器-解码器（CED） 10
2.3 压缩稀疏注意力 2（CSA2） 11
2.4 高效架构扩展 13
2.5 优化 15

第 3 章 通用基础设施 **18**
3.1 训练基础设施 18
3.2 推理系统 20

第 4 章 预训练 **22**
4.1 数据构建 22
4.2 预训练设置 22
4.3 评估 24

第 5 章 后训练 **27**
5.1 后训练流水线 27
5.2 异步后训练基础设施 31
5.3 评估 33

第 6 章 结论、局限性与未来方向 **38**

参考文献 **39**

作者名单 **46**

附录 A 评测细节 **48**
A.1 脚手架配置 48
A.2 不同脚手架下的推理强度 48
A.3 不同推理强度下推理基准的详细结果 49

附录 B 推理强度控制中的指数词元惩罚 **51**

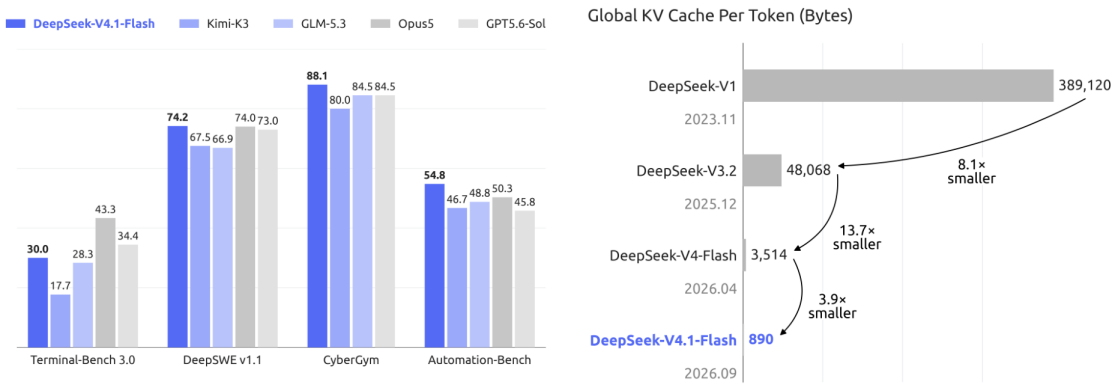


图 1 （a）DeepSeek-V4.1-Flash 与同类模型在智能体基准上的表现。（b）历代 DeepSeek 模型每词元的全局 KV 缓存大小（字节），凸显 DeepSeek 在降低上下文存储需求上的持续努力。相对于 DeepSeek-V4-Flash 与 DeepSeek-V1，DeepSeek-V4.1-Flash 的每词元全局 KV 缓存分别减少约 4 倍与 437 倍。

第 1 章 引言

近年来，长程智能体（long-horizon agent）应用迅速扩展，超长上下文处理逐渐成为日益重要的模型负载。支撑这类负载不仅需要高效的长序列处理，还需要对大型 KV 缓存进行持久存储、复用与传输。因此，KV 缓存管理已成为模型部署的一项基础能力，同时在计算、存储与通信三方面带来了显著挑战。稀疏注意力（sparse attention）方面的既有进展（DeepSeek-AI, 2025, 2026b）大幅降低了长序列处理的计算成本，使持久存储与数据搬运逐渐上升为更加突出的瓶颈。

具体而言，DeepSeek-V4（DeepSeek-AI, 2026b）将覆盖全部上下文的全局注意力分支与局部滑动窗口注意力（Sliding-Window Attention, SWA）相结合。全局分支维护全局 KV，包含主 KV（main KV）与索引器 K（indexer K），SWA 则维护局部 KV 状态。在窗口大小固定时，SWA KV 的存储量不受序列长度影响，始终有界。因此对于足够长的序列，全局 KV 将主导运行态 KV 占用，而后者受 HBM 容量的约束。此外，一部分 KV 会被持久保存以供前缀复用，称为持久 KV 缓存（persistent KV cache），其规模受 SSD 与主机内存容量的约束。I/O 与互连带宽同样限制缓存的迁移与加载。上述约束共同限制了服务吞吐，推高部署成本，并最终阻碍了智能体在更长任务周期、更广应用场景中的部署与采用。

因此，进一步压缩 KV 缓存占用，对缓解存储与通信瓶颈、降低长上下文服务成本至关重要。为此，我们开发了 DeepSeek-V4.1-Flash——一款为更激进 KV 缓存压缩而设计的多模态混合专家（Mixture-of-Experts, MoE）模型。DeepSeek-V4.1-Flash 拥有 552B 主干参数，原生支持多模态输入，可容纳最多一百万词元（token）的上下文。我们采用因果编码器-解码器（Causal Encoder-Decoder, CED）架构，解码器的全局 KV 由编码器最终的隐藏状态投影而来。该设计使模型在预填充（prefill）阶段每个词元仅激活 8B 参数，在解码（decode）阶段激活 16B 参数，对输入量偏大的智能体场景尤其划算。尽管规模显著大于 DeepSeek-V4-Flash，DeepSeek-V4.1-Flash 在相同序列长度下所需的运行时 KV 缓存存储仅为其约 1/4，持久 KV 缓存存储仅为其约 1/8。而且，DeepSeek-V4.1-Flash 的整体性能还优于 DeepSeek-V4-Flash。

这一程度的 KV 缓存压缩，得益于在模型架构、缓存精度与部署策略上的联合优化。从概念上看，DeepSeek-V4 可视为一个以 SWA 为基础的局部处理主干，再叠加经过压缩的全局上下文。这一视角促使我们将工作重心放在简化全局分支上，同时基本保留局部注意力设计。在架构层面，我们设计了压缩稀疏注意力 2（Compressed Sparse Attention 2, CSA2），对全局 KV（包括主 KV 与索引器 K）和 Top-K 索引施加跨层复用，从而大幅减少 KV 缓存存储。CSA2 设有三种静态分配的模式：Full、Reindex 与 Reuse。Full 模式生成全局 KV 并执行索引。Reindex 模式复用前一层产生的全局 KV，并用自身的索引器 Q 对共享索引器 K 重新打分，以选出新的 Top-K 索引。Reuse 模式则同时复用前一层的全局 KV 与 Top-K 索引，直接执行稀疏注意力。在全部三种模式下，每层都保留各自的全局 Q 与 SWA KV。共享全局 KV 与索引器 K 消除了重复的缓存存储。此外，DeepSeek-V4 采用压缩稀疏注意力（Compressed Sparse Attention, CSA）与重压缩注意力（Heavily Compressed Attention, HCA）的混合架构，而 DeepSeek-V4.1-Flash 与之不同，仅使用纯 CSA2。在缓存精度层面，我们在训练中即采用 FP4 全局 KV 缓存，性能退化微乎其微。如图 1(b) 所示，CSA2 与 FP4 KV 缓存二者结合，将全局 KV 缓存存储降至 DeepSeek-V4-Flash 的约 1/4。在部署层面，DeepSeek-V4.1-Flash 与 DeepSeek-V4 一样，每一层都使用滑动窗口注意

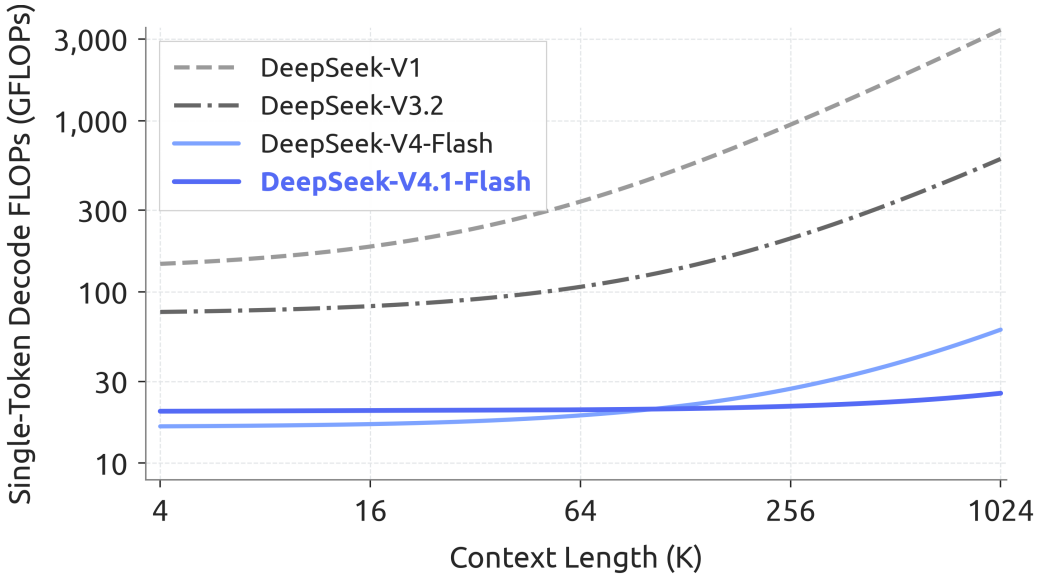


图 2　各代 DeepSeek 模型的单词元解码 FLOPs 随上下文长度的变化。我们对 BF16、FP8 与 FP4 运算分别按 1、0.5 和 0.25 加权，以计入计算精度的差异。随着上下文长度增加，DeepSeek-V4.1-Flash 的解码 FLOPs 几乎保持恒定，显著降低了长上下文场景的计算成本。

力（SWA）。在 DeepSeek-V4 中，我们采用一种混合策略，在持久化 SWA KV 缓存的存储开销与精确重建所需的计算之间取得平衡。精确重建需要重放最近的 $L \times n_{\text{win}}$ 个词元，其中 L 为层数，n_{win} 为 SWA 窗口大小。在 DeepSeek-V4.1-Flash 中，我们提出 SWA 有界重放（SWA Bounded Replay），仅重放最近的 n_{win} 个词元，即可近似重建所需的 SWA KV 状态。实验表明，这样做带来的性能退化可以忽略不计。这一发现确立了一种新的存储-计算权衡：只需在预填充阶段付出少量重计算，即可避免将 SWA KV 缓存持久保存到 SSD。借助 SWA 有界重放，持久 KV 缓存占用进一步降至 DeepSeek-V4-Flash 的约 1/8。这些优化共同作用，大幅缓解了 HBM 与 SSD 容量的压力，降低了部署成本，并为更大规模的部署铺平了道路。

作为对 CED 与 CSA2 的补充，我们还进一步精简了 DeepSeek-V4 的原有架构。我们将原有的 *m*HC（Xie et al., 2026）设计升级为单遍 *m*HC（Single-Pass *m*HC），并配套 Mega-*m*HC 部署算子，相比原先的四算子实现将激活内存流量减半。此外，我们集成了 Engram（Cheng et al., 2026b）条件记忆模块以增强模型能力，并引入 DSpark（Cheng et al., 2026a）推测解码架构，通过半自回归草稿生成与置信度调度的验证来提升解码效率。综合所有这些架构改进，DeepSeek-V4.1-Flash 的单词元解码 FLOPs 在各上下文长度下几乎保持恒定。图 2 显示，将上下文长度从 4K 扩展到 1M，即扩大 256 倍，其解码 FLOPs 仅增加 1/4，显著低于 DeepSeek-V4-Flash 的增幅。

为了充分兑现这些架构设计带来的 KV 缓存压缩收益，并进一步提升训练与推理效率，我们对 DeepSeek-V4.1-Flash 的训练基础设施与推理系统进行了系统性协同优化，确保大规模多模态训练与长上下文部署高效且可扩展。训练基础设施支持视觉编码器分离式执行、面向长序列的图像均衡切分，以及面向注意力复用的跨阶段共享状态管理。推理系统实现了编码器与解码器两条 SWA 有界重放路径。进一步的优化包括通信-计算重叠、分片的 Engram 嵌入表以及推理算子融合。特别地，每个 CSA2 Reuse 模式层在预填充阶段仅需 15 个算子执行，解码阶段仅需 11 个。我们还在主机内存中将长寿命的全局 KV 存储与短寿命的编码器 SWA KV 分开存放，并利用有界重放近似重建缺失的编码器 SWA 状态。

预训练阶段，我们在一个包含 45T 词元的大规模多模态语料上训练 DeepSeek-V4.1-Flash。稀疏注意力从零开始训练，序列长度为 64K，未经过任何稠密注意力预热阶段。预训练完成后，模型具备原生多模态能力，并支持最多一百万词元的上下文。在我们的评测中，DeepSeek-V4.1-Flash-Base 在世界知识、推理与编程能力上可比肩 DeepSeek-V4-Pro-Base，且仅使用 1/3 的总参数与 1/4 的激活参数，就在留出评测（held-out evaluation）上取得 5%-10% 的提升。这些结果共同凸显了其强劲的参数效率，也反映出面向真实部署的训练数据质量的改进。

在这一基座模型之上，我们进行后训练，以激发其推理与智能体能力。与上文所述的架构创新不同，我们的后训练并未引入算法上的创新：方案遵循监督微调（supervised fine-tuning, SFT）、随后强化学习（reinforcement learning, RL）与在策略蒸馏（on-policy distillation, OPD）的标准范式，除 DeepSeek-V4 开发（DeepSeek-AI, 2026b）中已被充分验证的做法外没有任何改动。所有实质性变化都落在数据管线上。我们开发了大规模自动化的数据合成与环境构建管线，并逐步扩大 RL 阶段所使用的数据、任务与轨迹采样（rollout）的规模，从而将模型能力延伸到文本、多模态与智能体等多个领域。图 1(a) 汇总了 DeepSeek-V4.1-Flash 在核心智能体基准上的表现。我们的评测显示，尽管体量紧凑，DeepSeek-V4.1-Flash 呈现出一组与众不同的能力特征：

- **Reasoning（推理）**：模型具备强大的推理能力，在数学、竞赛程序设计等以推理为核心的基准上保持高准确率，表现与 Kimi-K3（Team et al., 2026a）和 DeepSeek-V4-Pro 等顶尖开源模型相当。
- **Agent（智能体）**：在 Terminal-Bench 2.1（Merrill et al., 2026）、DeepSWE v1.1（DataCurve, 2026）与 AutomationBench（Shepard and Salimans, 2026）等标准智能体基准上，DeepSeek-V4.1-Flash 的性能与闭源前沿模型持平，并已被证明完全胜任日常编程任务与白领工作流。不过，在 Terminal-Bench 4.0（Marten et al., 2026a）等需要专家级领域知识的科学向智能体任务上，与巨型模型相比仍存在差距。
- **Multimodal（多模态）**：在多模态领域，模型在评测视觉推理与专业图表解读的基准上超越了 Kimi-K3 等一流开源竞争对手；除正式指标之外，它还在前端开发、办公自动化等真实世界的视觉智能体工作流中展现出实用价值，在这些场景中，它能利用渲染得到的屏幕截图进行视觉检查与自我纠错。不过我们也承认，与巨型闭源系统相比，整体性能差距依然明显。

这些结果表明，DeepSeek-V4.1-Flash 已能在绝大多数基准上与闭源前沿模型比肩，并能够完成超过 95% 的真实任务。同时，其很小的激活开销带来较低的推理延迟与服务成本。因此，我们相信 DeepSeek-V4.1-Flash 在能力与效率之间提供了有利的权衡，可以充当一款快速而经济的助手，支撑日常事务的处理，惠及广大用户群体。

……总体而言，DeepSeek-V4.1-Flash 在降低部署成本的同时，同步提升了模型智能与推理效率。它大幅降低了规模化部署长程智能体（long-horizon agents）的成本门槛，为其在更广泛的场景中落地创造了新的机会。DeepSeek-V4.1-Flash 也为我们持续的规模化探索提供了新的起点。在此基础之上，我们将推进模型架构、预训练与后训练的联合扩展，继续探索模型智能的前沿。

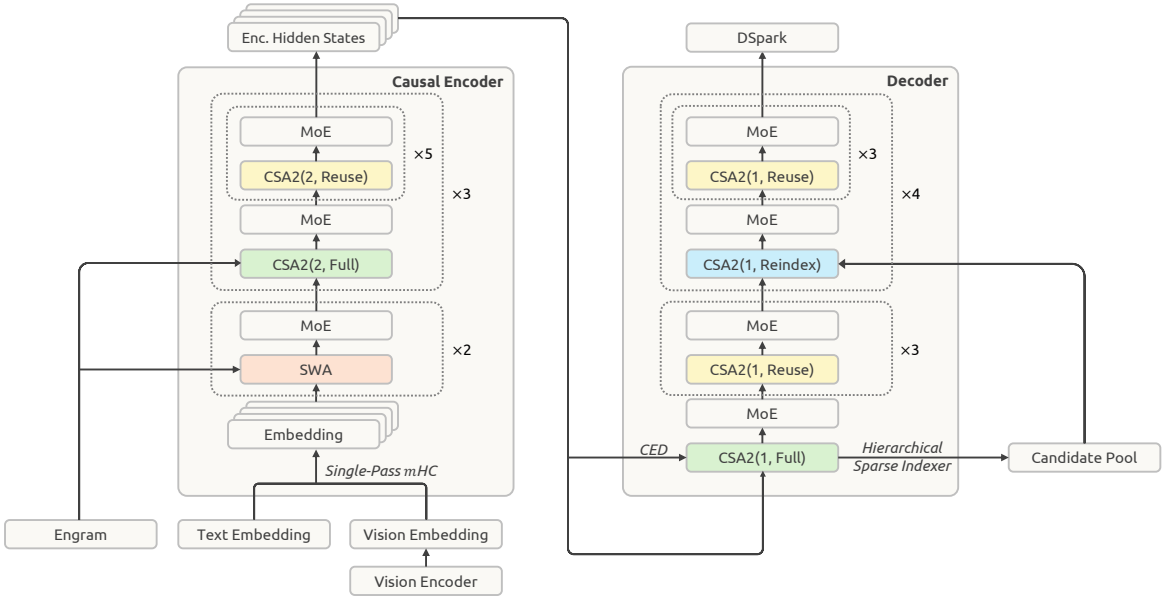


图 3　DeepSeek-V4.1-Flash 整体架构。40 层网络划分为一个因果编码器和一个解码器，各含 20 层。所有前馈层均采用标准 DeepSeekMoE。编码器的前两层使用滑动窗口注意力（SWA），其余层使用 Compressed Sparse Attention 2（CSA2），由 CSA2(*ratio*, *mode*) 指定压缩比与模式。模型还采用 Single-Pass *m*HC、Engram、DSpark 以及一个分层稀疏索引器（Hierarchical Sparse Indexer）。

第 2 章 架构

2.1 概述

DeepSeek-V4.1-Flash 是一个多模态混合专家（Mixture-of-Expert，MoE）Transformer，以图像和文本作为输入，并以自回归方式生成文本。其语言主干由 40 层因果 Transformer 层组成，组织为一个 20 层因果编码器后接一个 20 层解码器。除前两层仅使用滑动窗口注意力（SWA）之外，其余每一层均同时包含全局注意力与滑动窗口注意力（SWA）。视觉编码器与 MLP 投影器将图像转换为视觉嵌入，这些嵌入与文本嵌入一同处理，多模态数据从语言模型预训练的最初阶段即已纳入。总体上，DeepSeek-V4.1-Flash 拥有 552B 主干参数和 196B Engram 参数，预填充阶段每个词元（token）激活 8B 参数，解码阶段激活 16B 参数。图 3 展示了 DeepSeek-V4.1-Flash 的整体架构。

因果编码器—解码器（Causal Encoder-Decoder，CED）架构与 Compressed Sparse Attention 2（CSA2）分别解决长上下文推理中互补的两类开销。CED 从编码器输出构造解码器的全局键值（KV）缓存，使大多数提示词元（prompt token）无需经过完整的解码器计算，同时保留层内局部的滑动窗口注意力。这使预填充计算量近乎减半，降低了上下文不断增长的智能体工作负载中处理新输入或未缓存输入的成本。CSA2 跨层共享全局 KV 以减少缓存存储，并通过复用稀疏选择结果来减少索引工作量。在解码器中，分层稀疏索引器（Hierarchical Sparse Indexer）把后续索引器限制在由更早的索引器选出的候选池内，进一步减少了每个查询（query）需要打分的条目数量。

我们保留了 DeepSeekMoE（Dai et al., 2024）中的共享专家与细粒度路由专家，并针对图像与文本词元引入模态特定的负载均衡（modality-specific load balancing，Wang et al., 2024a）。Single-Pass *m*HC（Xie et al., 2026）对残差流混合（residual-stream mixing）方式作了修改，以实现更高效的核融合（kernel fusion）；Engram（Cheng et al., 2026b）则增加了稀疏访问的条件记忆。主干预训练期间我们省略 MTP 模块，并使用 DSpark（Cheng et al., 2026a）进行轨迹采样式投机解码。DSpark 在主干预训练阶段之后单独训练。此外，我们将主 KV 缓存压缩至 FP4，以进一步降低存储开销。后续各节将介绍这些组件以及相应的优化改动。

2.1.1 多模态架构

多模态输入通路由视觉编码器与 MLP 投影器组成。对每张输入图像，视觉编码器生成空间网格状排布的视觉特征。随后，一次 3×3 像素重排（pixel-unshuffle）操作沿通道维重新排列每个局部邻域，降低空间分辨率；之后 MLP 投影器再把特征映射到语言主干的隐藏维。最后，所得视觉嵌入被插入输入嵌入序列中对应的图像词元位置，与文本嵌入一起由语言主干处理。

DeepSeek-ViT 我们从零开始训练一个名为 DeepSeek-ViT 的视觉编码器，使其能原生处理不同分辨率的图像。DeepSeek-ViT 基于 Vision Transformer（Dosovitskiy et al., 2021）架构构建，并作了若干修改。为容纳任意分辨率的输入，我们将标准的绝对位置嵌入替换为 2D-RoPE。为使 ViT 更贴合 LLM 的设计原则，我们将 patch 嵌入层的卷积替换为线性投影，以保证与 Muon 优化器兼容。归一化采用 RMSNorm（Zhang and Sennrich, 2019），激活函数采用 SwiGLU（Shazeer,

2020）。在把视觉特征送入 LLM 之前，我们执行一次 3×3 下采样的像素重排（pixel-unshuffle）操作，使视觉词元数量缩减至原来的九分之一，等效支持约 1344×1344 像素的输入分辨率。

MoE 的多模态无辅助损失负载均衡（Multimodal Auxiliary-loss-free Load Balancing for MoEs）图像与文本词元具有不同的表示分布，可能在 MoE 中引发不同的专家路由偏好。对二者的聚合负载作均衡，反而可能掩盖模态内部的失衡。为解决这一问题，我们对无辅助损失负载均衡（auxiliary-loss-free load balancing，Wang et al., 2024a）加以扩展，为文本与图像词元分别维护一组专家级修正偏置。路由时，每个词元使用其所属模态的修正偏置进行专家选择，而所选专家输出的加权仍使用原始路由分数。每个训练步结束后，两组偏置依据各自覆盖的专家负载独立更新。这一设计在每种模态内部均衡了专家利用率，有助于多模态训练的稳定与高效。

2.2 因果编码器-解码器（CED）

在智能体工作流中，频繁的工具调用会产生大量预填充请求，一旦 KV 缓存缺失，计算开销会非常沉重。为缓解这一预填充瓶颈，我们在 YoCo（Sun et al., 2024）的启发下提出因果编码器-解码器（Causal Encoder-Decoder, CED）架构。YoCo 让上半部分的层直接共享下半部分生成的 KV 缓存，从而减少预填充计算。在这一思路的基础上，CED 引入了一系列结构性改进，同时提升 KV 缓存的整体容量和 KV 生成的计算深度。最终，CED 在保持与基线相当的性能的同时，将预填充计算量削减了近一半。

对于全局注意力，CED 把 Transformer 底部的 $L/2$ 层视为因果编码器。对于上半部分的各层（即解码器，$l > L/2$），其键值项不再由各自的隐藏状态 H_l 计算得到，而是直接从第 $L/2$ 层的隐藏状态投影而来，即

$$C_l = H_{L/2} W_l^{KV}, \qquad Z_l = H_{L/2} W_l^{Z}, \qquad l > \frac{L}{2}, \tag{1}$$

其中，C 和 Z 分别表示键值项及其对应的压缩权重。这一设计使 CED 在预填充阶段只需计算前一半的层，便能以极小的计算代价获得上层的全局 KV 缓存。

对于滑动窗口注意力（SWA），CED 在所有层上都保留传统的逐层计算。具体而言，对任意一层 l，局部键和值直接从该层的隐藏状态 H_l 导出。这一设计有效加深了局部 KV 生成的计算深度。但要在所有层上维持逐层计算，就必须执行一个 SWA 重放（replay）过程：预填充阶段为解码器计算 SWA KV 缓存时，需额外处理 $n_{\text{win}} \times L/2$ 个词元（n_{win} 表示窗口大小）。在每轮提示都很短的多轮交互中，解码器的这部分计算开销不可忽略。所幸已有工作（Chen et al., 2025）表明，SWA 的实际有效感受野远小于理论上的 $n_{\text{win}} \times L/2$。受此观察启发，我们提出 Decoder SWA Bounded Replay，只为 SWA 计算预填充提示的最后 n_{win} 个词元，从而显著降低计算成本。进一步细节见第 3.2.2 节。

总体而言，当序列长度 $N \gg n_{\text{win}}$ 时，CED 把预填充复杂度从 $O(NL)$ 降低到 $O(NL/2 + n_{\text{win}} \times L/2) \approx O(NL/2)$，相当于把整体计算量减半。

2.3 压缩稀疏注意力 2（CSA2）

服务长上下文需要同时控制 KV 缓存存储与注意力计算两类开销。这两类开销可以沿三个相乘的维度压减：条目大小维度，GQA（Ainslie et al., 2023）减少 KV 头的数量，MLA（DeepSeek-AI, 2024）让各头共享一个很小的潜在向量；序列维度，每 m 个词元被压缩为一个条目，例如 DeepSeek-V4（DeepSeek-AI, 2026b）中的 CSA 与 HCA；层维度，一些层不再保留自己的缓存，而是复用其他层的缓存（Brandon et al., 2024）与选择结果，或者干脆被更高效的层彻底替换。已有工作表明，沿层维度的压缩是有效的：IndexCache（Bai et al., 2026）跨层复用 Top-K 索引以削减索引器计算；YOIO（Sun et al., 2026b）只计算一次稀疏路由并在所有层间共享；HySparse（Gao et al., 2026）让稀疏层复用稠密层的 KV 缓存。然而，仅复用索引省不下任何主 KV 存储；全网共享路由会限制性能；混合设计也仍保留完整的注意力层。更重要的是，这些方法没有一个同时覆盖全部三个相乘维度。

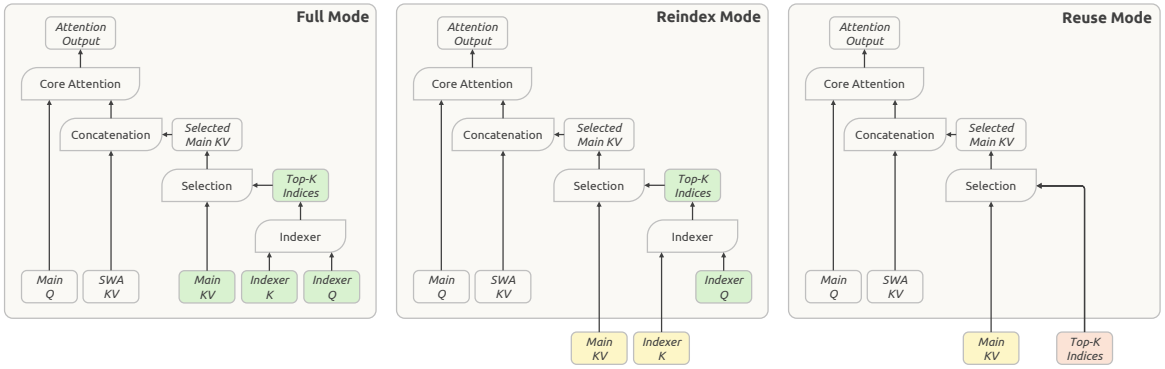


图 4　CSA2 的三种运行模式。各模式的区别在于主 KV、索引器 K 与 Top-K 索引的获取方式。绿色块表示在当前层计算的量；黄色块表示复用自最近一个完整模式（Full Mode）层的主 KV 与索引器 K；红色块表示复用自最近一个产生索引的层（完整或重索引模式）的 Top-K 索引。三种模式都在当前层计算主 Q 与 SWA KV。

CSA2 同时利用这三个维度：它让主 KV 与索引器 K 跨层共享，并允许各层复用 Top-K 索引，且缓存共享与索引复用相互解耦。CSA2 把这些复用策略与一个简化后的压缩器以及一个层级稀疏索引器（Hierarchical Sparse Indexer）结合在一起，后者缩小了解码器中后续索引层的搜索域。

与 CSA 类似，CSA2 也包含一个轻量级索引器：它用索引器 Q 与索引器 K 对主 KV 条目打分，并为每个查询选出 Top-K 条目。每个查询在所选条目与该层局部的滑动窗口 KV（SWA KV）上做注意力。压缩比为 1、即不对主 KV 做压缩的设定也被 CSA2 作为特例包含在内。与此同时，CSA2 对压缩器与索引器都做了简化。在 CSA 中，压缩比 m 表示每个主 KV 条目由 $2m$ 个原始 KV 缓存条目生成，相邻压缩条目的源条目相互重叠；压缩时还引入绝对位置嵌入来编码这 $2m$ 个条目的位置。CSA2 去除了这种重叠与绝对位置嵌入。此外，CSA2 通过对主 KV 条目做投影得到索引器 K，取代了 CSA 从隐藏状态单独压缩的路径。这两处设计都简化了实现并提高了训练效率。

第 2.3.1 节与第 2.3.2 节分别介绍跨层复用策略与层级稀疏索引器。

2.3.1 跨层 KV 与索引复用

每个 CSA2 层被静态指派为三种模式之一：完整模式（Full Mode）、重索引模式（Reindex Mode）或复用模式（Reuse Mode）。三种模式下，该层都计算自己的查询与 SWA KV，并把它们与选出的主 KV 条目一起用于生成新的注意力输出。三种模式的差别在于获取主 KV、索引器 K 与 Top-K 索引的方式。图 4 展示了这三种模式。

完整模式。该层自己计算主 KV 与索引器 Q，从该主 KV 投影得到索引器 K，再运行索引器产生新的 Top-K 索引。因此该层执行完整的 CSA2 计算路径，各组件的职责与 DeepSeek-V4 中一个完整的 CSA 层相同。

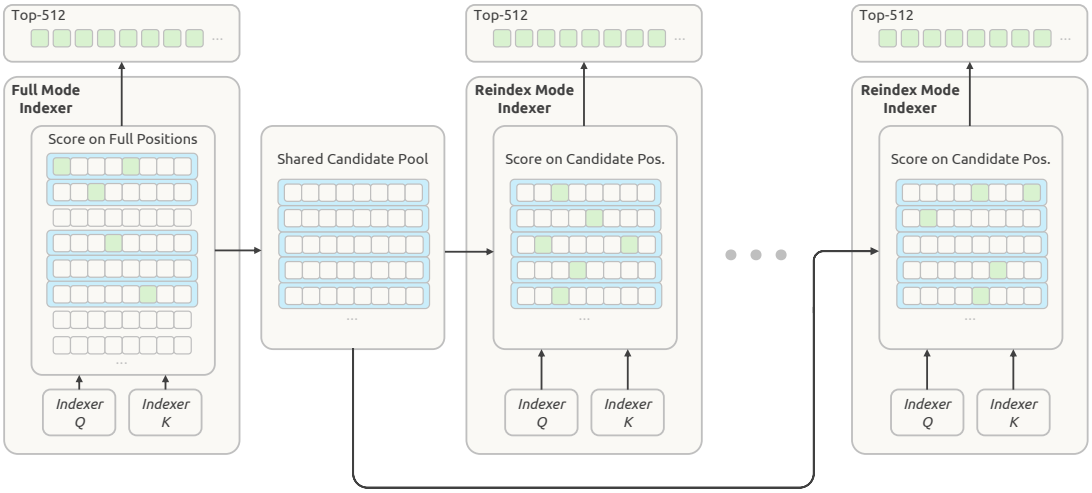


图 5　层级稀疏索引器。每个方块代表一个位置；绿色方块标出被选中的索引，蓝色矩形标出依据索引器最高分数选出的块。解码器中第一个处于完整模式的 CSA2 层会选出自己的 Top-512 索引，并基于所选块为后续各层构建一个共享候选池；处于重索引模式的 CSA2 层随后从这个池中选出各自的 Top-512 索引。

重索引模式。该层复用前一层最近可用的主 KV 及其对应的索引器 K。索引器计算自己的查询，对复用的键重新打分，并产生新的 Top-K 索引。这样，在主 KV 与索引器 K 保持共享的同时，稀疏选择可以跨层变化。

复用模式。该层复用前一个完整模式或重索引模式层最近可用的主 KV，以及针对该主 KV 算出的最新 Top-K 索引。它直接用这组选择做注意力，既不计算索引器 Q，也不评估索引分数。

共享主 KV 与索引器 K 可以减少缓存存储，复用 Top-K 索引则省去额外的索引器计算。重索引模式在保持缓存共享的同时，允许被选条目跨层变化。当 CSA2 与 CED 组合时，被指派为完整模式的解码器层从第 $L/2$ 层（即因果编码器的最后一层）的隐藏状态计算自己的全局 KV。重索引模式与复用模式保持不变。

2.3.2 层级稀疏索引器

跨层索引复用减少了索引器求值的次数，但余下的索引器仍要对完整因果可见上下文打分。对极长上下文而言，这部分开销仍是主要的计算瓶颈。已有工作在词元级索引之前，先对池化后的块表示打分并剪枝，从而引入索引器稀疏性（Xu et al., 2026b）。我们发现，在解码器中，较浅索引器的信息可以自然地用来收窄较深索引器需要考虑的候选范围，而不引入任何额外状态。

为此，我们提出层级稀疏索引器，它只用于 CED 的解码器，以减少解码阶段这种重复打分。对每个查询，第一个被指派为完整模式的层会构建一个候选池，后续执行重索引的层把它当作搜索域。在候选池大小固定的前提下，这使较深索引器的单查询开销从随上下文长度线性增长变为

保持不变。该机制是训练感知的（training-aware），并在后训练阶段引入：候选限制在训练与推理中以完全相同的方式施加，从而更深的索引器在与推理时相同的搜索域内得到优化。图 5 展示了这一过程。

第一个 Full Mode 层对所有因果可见的主 KV 位置打分，并为其自身的注意力产生 Top-K 索引。它还执行按块的候选选择：每个块取其所在位置中的最大索引分数，并选取得分最高的若干块。随后，它把被选块覆盖的位置汇入一个比最终 Top-K 集合更大的候选池。例如，按每块 8 个位置选中 2,048 个块，即可得到 16,384 个候选位置。该候选池界定了后续索引器的搜索范围；而最终的 Top-K 选择决定每一层读取哪些主 KV 条目。

处于 Reindex Mode 的后续层只对相应查询的候选位置打分，并从该候选池中选出各自的 Top-K 条目。处于 Reuse Mode 的层不执行新的索引，而是直接使用最近一次针对其所复用的主 KV 计算出的 Top-K 索引。因此，候选池在各索引层之间共享，而各层的最终选择可以不同。

在候选池大小固定的条件下，每个后续索引器对单个查询所需打分的位置数量有一个与上下文长度无关的上界。第一个 Full Mode 层仍要扫描整个因果可见范围。因此，分层索引在保留初始全范围扫描的同时，降低了后续索引器评估的开销。

2.4 高效架构扩展

2.4.1 单趟 mHC

在 DeepSeek-V4 中，我们引入了 mHC（Xie et al., 2026），它在相邻 Transformer 块之间维护 n 条残差流。对每个词元，我们把这些流记作 $X_l \in \mathbb{R}^{n\times d}$，其中 l 是块索引，d 是隐藏维度。这些流按下式更新：

$$X_{l+1} = B_l X_l + C_l\, \mathcal{F}_l(A_l X_l), \qquad (A_l, B_l, C_l) = \mathcal{H}(X_l), \tag{2}$$

其中系数预测器 $\mathcal{H}$ 包含归一化和投影。

理想情况下，相邻两个块之间的残差变换应当是 (X_{l-1}, Y_{l-1}) 到 $(X_l, \hat{X}_l)$ 的单一映射，其中 $\hat{X}_l = A_l X_l$ 是当前块的输入，$Y_{l-1} = \mathcal{F}_{l-1}(\hat{X}_{l-1})$ 是前一个块的输出。这样的映射需要 $(n+1)d$ 次读和 $(n+1)d$ 次写，从而给出激活内存流量的下界 $(2n+2)d$。实践中，DeepSeek-V4 采用式 (2) 的多趟实现，由三个内核组成，它们因数据依赖而必须顺序执行：

$$\hat{X}_l = A_l X_l \tag{5}$$

这三个内核分别读取 $(n+1)d$、nd、nd 个数值，合计写入 $(n+1)d$ 个数值。计入 $\mathcal{F}_l$ 中的前置归一化（pre-norm）后，激活内存流量总额为 $(4n+4)d$，是下界的两倍。在 $\mathcal{H}$ 中，归一化权重在离线阶段被折叠进投影权重，而 RMS 除法则在投影之后施加。因此，三个阶段中有两个可

以共用对残差的一次遍历：残差更新不需要沿隐藏维度做归约，所以 X_l 的每个分块（tile）都可以直接计算出来，并立即用于累加投影输出以及计算 RMS 所需的平方和。输入混合无法融合进这一趟，因为 A_l 要等到所有隐藏分块的归约完成后才可用，这就迫使输入混合对 X_l 再一次读取。第二趟还可以顺便完成输入前置归一化。这种两趟实现总共需要 $(3n+2)d$ 的激活读与写，比下界多出对 X_l 的一次读取。

因此，我们提出单趟 mHC（Single-Pass mHC），它把输入混合系数移位一个块，即每个块消费的是前一个块产生的混合系数，从而消除上述依赖：

$$X_{l+1} = B_l X_l + C_l\, \mathcal{F}_l(A_{l-1} X_l), \qquad (A_l, B_l, C_l) = \mathcal{H}(X_l). \tag{6}$$

输入混合此时使用的是 A_{l-1} 而非 A_l，因此不再依赖由 X_l 计算出的系数。于是，X_l 的每个分块在算出后即可立即用于输入混合和系数预测，而无须等待完整归约。从实证结果看，这一移位带来的性能退化可以忽略。

预训练时，我们保留现有的多内核实现，因为这一移位只改变每个块所应用的混合系数。部署时，我们把残差更新、输入混合与系数预测融合为单一内核 Mega-mHC。该内核实现 mHC 需要 $(3n+2)d$ 的激活读与写，实现单趟 mHC 则为 $(2n+2)d$。Mega-mHC 沿隐藏维度按分块处理 X_l：每个分块既用于计算混合输入，也用于累加预测下一块 (A_l, B_l, C_l) 所需的量。该内核还纳入了输入前置归一化和 FP8 转换。残差因此只读一次、写一次，达到理想映射的 $(n+1)d$ 读与 $(n+1)d$ 写，并把原实现的激活内存流量减半。

2.4.2 Engram

我们用 Engram（Cheng et al., 2026c）增强了 DeepSeek-V4.1-Flash；Engram 是我们此前工作中提出的条件记忆模块，用于把记忆与计算解耦。我们沿用原版 Engram 设计——分词器压缩、多头哈希、上下文感知门控与多分支整合——仅做两处修改。其一，省去短因果卷积，因为其性能收益不足以抵偿推理技术栈中新增的复杂度。其二，对 Engram 嵌入采用基于动量的更新加 Sinkhorn 平衡进行优化，细节见第 2.5 节。

我们把共 196B 的 Engram 参数平均分配到两个模块。每个模块使用 N-gram 阶集 $\{2, 3, 4\}$，含 8 个哈希头，且每个阶数的总嵌入维度为 2048。每个头索引一张约 16M 条目的表，各表容量均选为互不相同的素数。嵌入表与 key/value 投影均使用 FP8 精度。两个模块布置在第 1 层和第 14 层（索引从 0 起算），以在各训练流水线阶段之间平衡内存占用。推理时，确定性寻址使得嵌入能够通过后台 RDMA 传输从主机内存预取，其中第一个模块的预取与第一个 Transformer 块的计算相重叠。Engram 训练与推理的进一步实现细节见第 3.1.3 节。

2.4.3 DSpark

我们为 DeepSeek-V4.1-Flash 配备了 DSpark（Cheng et al., 2026a）——一个把半自回归草拟（semi-autoregressive drafting）与置信度调度验证（confidence-scheduled verification）相结合的投机解码（speculative decoding）模块。

起草器由三个 Transformer 块组成，使用 128 个词元的滑动注意力窗口。对这些块做一次前向传播，即可并行算出五个草稿位置的基础 logits；一个轻量级马尔可夫头则建模草稿词元之

间的依赖关系。置信度头预测每个位置的条件接受概率，这些概率进而用于估计前缀存活概率。调度器将上述估计与实测得到的引擎吞吐曲线相结合，为每个请求动态选择验证长度，力求在当前系统负载下最大化全系统词元吞吐的期望。

DeepSeek-V3（DeepSeek-AI, 2024）的 MTP 模块在整个预训练期间与主干联合训练，DSpark 则不同：它在预训练结束后的一个专门阶段引入。该阶段只训练 DSpark，主干保持冻结。进入后训练阶段，我们继续连同主干一起训练 DSpark，但不把 DSpark 目标的梯度回传给主干。这使得 DSpark 与不断演化的策略保持一致，从而既能加速在线服务，也能加速 RL 与 OPD 的轨迹采样生成。

2.4.4 FP4 主 KV 缓存

长上下文智能体工作负载要求每请求维护很大的 KV 缓存，推高了服务成本。DeepSeek-V4 已对 FP4 索引器的查询与键采用量化感知训练（QAT）（Jacob et al., 2018），既加速索引计算，又缩小索引器缓存。尽管备选格式在我们的实验中精度更高，我们仍采用 OCP 标准的 MXFP4 格式（Rouhani et al., 2023），以便尽可能广泛地兼容各类硬件平台。现在，我们把 QAT 扩展到主 KV 缓存：在这里，FP4 的作用是压缩存储，而非加速矩阵乘法。在注意力计算前先对缓存值去量化，我们便可以使用精度更高的格式，而无需该格式具备原生矩阵乘法支持，从而保持跨硬件平台的兼容性。

在评估过的各种约四比特格式中，我们选择每 16 个通道配一个 E4M3 标度的 E2M1，遵循 NVFP4（Alvarez et al., 2025）的做法，但省去其第二级全局标度，以在精度与简洁之间取得平衡。省去该标度为主 KV 缓存留出了充足的动态范围：该格式可表示的最大幅值达 $448 \times 6 = 2688$，远高于缓存的幅值上界。在 DeepSeek-V4.1-Flash 中，训练所得 RMSNorm 权重的最大幅值约为 1。经 RMS 归一化后，512 通道 KV 潜在向量的 L2 范数至多约为 $\sqrt{512}$。RoPE 保持该范数不变，因此旋转后各通道的最大绝对值同样被 $\sqrt{512} \approx 22.6$ 约束。此外，训练期间观测到的最大幅值约在 10 左右。因此，省略全局标度不会造成可测量的精度下降，还简化了缓存布局。

为了在 DeepSeek-V4.1-Flash 中启用 FP4 主 KV 缓存存储，我们在后训练阶段引入 QAT。非 RoPE 分量与 RoPE 分量使用相同的量化格式。我们在 RoPE 之后对缓存做量化：实验表明，先量化再做 RoPE 只带来边际的精度提升，却会在解码时引入额外开销。鉴于 SWA KV 缓存对量化敏感，我们为其保留 FP8。与 DeepSeek-V4 的 FP8 主 KV 缓存相比，该格式几乎将存储占用减半，无论数据驻留在 HBM 中还是被卸载到 SSD 上。

2.5 优化

我们在 DeepSeek-V4 所用优化配置的基础上做了若干新的修改，使其与架构设计更加契合。

第一，我们采用按头 Muon（head-wise Muon）：在应用 Muon 更新之前，先将 Query 权重按注意力头拆分。这里我们简要说明这一设计的动机。把 Muon 视为带预条件器的梯度下降，原始 Muon 对所有头共用同一个预条件器，而按头 Muon 则为不同头提供各自的预条件器。这一设计能更好地处理各注意力头之间的异质性（Zhang et al., 2024; Zhang, 2026, Section 3）。在实际效果上，我们观察到按头 Muon 优于原始 Muon。GLM 5（Zeng et al., 2026）与 Kimi-K3（Team et al., 2026a）也验证了按头 Muon 的经验优势。

第二，对新引入的 Engram 参数使用 Adam 会显著增加优化器状态的内存占用。为了降低训练内存，我们转而采用基于动量的更新加 Sinkhorn 平衡来优化 Engram 嵌入表、词元嵌入和预测头。SinkGD（Scetbon et al., 2025）此前已把 Sinkhorn 平衡应用于线性层权重矩阵；这里我们把它扩展到这些大型参数矩阵。与 Muon 一样，该方法只需维护一个动量缓冲区，同时在经验上优于 Adam。

基本配置。归一化层权重以及偏置、缩放因子等其他非矩阵参数，仍用 AdamW（Loshchilov and Hutter, 2019）优化。语言模型主干中线性变换的权重矩阵、Engram 投影层以及视觉-语言投影器，用 Muon（Jordan et al., 2024）优化。Query 权重与 Key 权重则用按头 Muon。我们对 Muon 应用解耦的权重衰减与 Nesterov 动量（Nesterov, 1983; Liu et al., 2025）；归一化层权重同样施加权重衰减，而偏置与缩放因子不施加。Sinkhorn 平衡更新同样使用 Nesterov 动量，但不做权重衰减。预训练期间，视觉编码器在学习率衰减阶段开始前保持冻结，其最后的归一化层与视觉-语言投影器保持可训练。学习率衰减开始时，我们解冻视觉编码器，并以较小的学习率将其与 LLM 联合优化。

面向 Engram、嵌入与预测头的 Sinkhorn 平衡更新。完整流程总结在算法 1 中。整体上，它遵循与 Muon 相同的工作流程，只是以 Sinkhorn 平衡取代 Newton-Schulz 正交化。我们把矩阵较大的维度记作 m——对嵌入表和预测头而言，它对应词表大小；并把隐层维度记作 n。给定 Nesterov 动量更新 $\hat{G}_t$，Sinkhorn 平衡寻找对角缩放矩阵 D_r 与 D_c，

$$\Delta_t = \sqrt{n}\, U^{(K)} = \sqrt{n}\, D_r \hat{G}_t D_c, \qquad \frac{1}{n}\sum_{j=1}^{n}(\Delta_t)_{ij}^2 \approx 1, \qquad \frac{1}{m}\sum_{i=1}^{m}(\Delta_t)_{ij}^2 \approx 1, \tag{7}$$

于是，该过程近似地均衡了更新矩阵的行 RMS 与列 RMS。这里，一行对应一个词元索引或一个 N-gram 标识，一列编码一个隐层特征。Sinkhorn 平衡正是利用这种词元-特征结构，同时沿行和列做归一化。出于数值稳定性考虑，满足 $\rho_i \le \tau\bar{\rho}$ 的行会被掩蔽。因子 $\sqrt{n}$ 把单位行 ℓ_2 范数换算为单位行 RMS。另外，我们把有效学习率调整为 $\tilde{\eta}_t = \gamma\eta_t$，以匹配 Adam 的更新幅度。我们取 $\gamma = 0.18$，接近 Moonlight（Liu et al., 2025）中使用的 0.2。

更广泛地说，Sinkhorn 平衡与利用矩阵或张量轴结构的优化器密切相关（Shazeer and Stern, 2018; Zhang et al., 2025a; Wen et al., 2025; Glentis et al., 2025; Deng et al., 2026; Yuan et al., 2026; Xu et al., 2026a）。例如，Adafactor（Shazeer and Stern, 2018）

算法 1（带 Sinkhorn 平衡的动量更新）

输入：权重 $W_t \in \mathbb{R}^{m\times n}$，梯度 G_t，动量 M_{t-1}，动量系数 β，基础学习率 η_t，学习率修正 γ，数值常数 ε 与 τ，以及奇数个归一化步数 K。

$M_t \leftarrow \beta M_{t-1} + (1-\beta)G_t$

$Gb_t \leftarrow \beta M_t + (1-\beta)G_t$　（Nesterov 动量）

$\rho_i \leftarrow \|G_{t,i,:}\|_2$，$\bar{\rho} \leftarrow \frac{1}{m}\sum_{i=1}^{m}\rho_i$

$U \leftarrow G_t$

当 $\rho_i \leqslant \tau\bar{\rho}$ 时置 $U_{i,:}^{(0)} \leftarrow 0$　（屏蔽近零行）

for $k = 1, \dots, K$ **do**

if k 为奇数 **then**

for all $i = 1, \dots, m$ **do**

$U_{i,:}^{(k)} \leftarrow U_{i,:}^{(k-1)} / (\|U_{i,:}^{(k-1)}\|_2 + \varepsilon)$

10: **end for**

11: **else**

12: **for all** $j = 1, \dots, n$ **do**

13: $U_{:,j}^{(k)} \leftarrow U_{:,j}^{(k-1)} / (\|U_{:,j}^{(k-1)}\|_2 + \varepsilon)$

14: **end for**

15: **end if**

16: **end for**

17: $\Delta_t \leftarrow \sqrt{n}\, U^{(K)}$ （把单位行 ℓ_2 范数换算为单位行 RMS）

18: $\tilde{\eta}_t \leftarrow \gamma \eta_t$ （使更新幅度与 Adam 匹配）

19: $W_{t+1} \leftarrow W_t - \tilde{\eta}_t \Delta_t$（Stern, 2018）用另一种方式执行按行与按列的归一化，Adam-mini（Zhang et al., 2025a）则对嵌入表与预测头采用另一种按行归一化。这些归一化策略在优化性能与通信开销上可能各有差异。我们把更细致的研究留作未来方向。

第 3 章 通用基础设施

3.1 训练基础设施

3.1.1 多模态训练基础设施

对比学习中的通信与计算重叠。视觉编码器先以对比目标进行优化，之后再以生成式的下一词元预测损失做微调。在对比阶段，损失是在一整批文本—视觉配对样本上计算的，因此两种模态的特征都必须跨数据并行 rank 进行 all-gather，通信开销相当可观。由于文本特征的梯度只依赖于汇聚后的视觉特征——对称地，视觉特征的梯度也只依赖于汇聚后的文本特征——每次 all-gather 都可以与前向或后向计算重叠，而不必让流水线停摆：

$$\cdots \;\rightarrow\; \text{Backward}(T) \,\|\, \text{AllGather}(T) \;\rightarrow\; \nabla\text{Vision} \;\rightarrow\; \text{Backward}(V),$$

其中 V 和 T 分别表示视觉特征与文本特征，$(A \,\|\, C)$ 表示把计算 A 与通信 C 重叠，∇ 表示梯度计算。在这一调度中，视觉特征在文本前向阶段被 all-gather，文本特征则在文本后向阶段被 all-gather，因此两次 all-gather 都被完全隐藏在有效计算之后。

端到端并行。为处理视觉编码器与 LLM 之间的模型和数据异构（Zhang et al., 2025b），我们采用近期训练系统（Team et al., 2025, 2026b）中使用的解耦编码器（disaggregated encoder）设计。视觉编码器在 LLM 参数树之外做副本部署，每个训练步分为三个阶段：视觉编码器前向、LLM 前向/后向、视觉编码器后向。这种隔离避免了视觉编码器与 LLM 计算之间的相互干扰。负载均衡的视觉处理只发生在第一个和最后一个阶段，LLM 阶段不含任何视觉计算，并保持纯文本训练的并行策略。

长序列多模态训练优化。DeepSeek-V4.1-Flash 在最长一百万词元的序列上训练，且在预训练与后训练两个阶段，相当大一部分训练发生在超长序列长度下。在这种序列长度下，多模态样本会带来沉重的 I/O、CPU 与内存瓶颈。

均衡的图像切分。预训练期间，一条超长且图像密集的序列在加载时就可能耗尽单台主机的 I/O、CPU 与内存，因此每条序列的图像会按负载均衡切分到各 CP rank 上，且每张图像只加载一次。图像只需读取一次，只要满足下式，加载就能完全隐藏在计算之后：

$$\frac{N \times \rho}{B_{\text{IO}}} \;<\; \frac{N \times C}{B_{\text{GPU}}} \quad\Longleftrightarrow\quad \rho < C,$$

其中 N 为词元数量，ρ 为每词元的原始字节数，C 为每词元的计算量，B_{IO} 与 B_{GPU} 分别为文件系统带宽与 GPU 带宽。由于 N 可以约去，该判据只涉及每词元的量（ρ 与 C），与序列长度和集群规模无关；ρ 由视觉模块的配置决定（例如分辨率上限或空间下采样）。因此，存储吞吐只在每词元计算量较低的小模型（如消融实验）中成为瓶颈，而生产规模的模型仍处于算力受限（compute-bound）状态。

增量式图像传输。除了上述均衡切分之外，强化学习轨迹采样只把图像增量式地传输给推理引擎，并把引擎在 CPU 侧的解码与预处理输出缓存在分布式文件系统上，供多次轨迹采样及后续训练复用。

3.1.2 面向 CSA2 的注意力共享训练

在第 2.3 节中，我们介绍了 CSA2，这是一种带三种模式的注意力共享方法，其中部分模式会让多个层共享主 KV、索引器 K（indexer K）以及 Top-K 索引中的一者或多者。要在大规模分布式训练中支持 CSA2，除了注意力计算本身之外，还需要额外的协调：共享注意力组件的各层可能被放到不同的流水线阶段上，这使直接的模块复用与常规的阶段局部执行（stage-local execution）不相容。因此，我们采用若干设计来支持 CSA2 训练。

影子索引器（shadow indexer）的解决方式是：在每个参与共享的阶段上部署一份轻量、可执行的副本，同时为共享参数只保留一个逻辑属主。属主始终负责优化与检查点保存，而参数同步　与梯度聚合机制让影子副本在整个训练过程中保持一致。这一设计保留了原模型的语义，而无需流水线调度器把共享层当作特殊的执行单元来处理。

流水线负载扩展在源层与消费层跨越流水线边界时，向下游消费者提供所需的中间表示与稀疏路由信息。这些状态被并入既有的点对点通信路径，并按与上下文并行一致的方式切分，在维持相应梯度流动的同时避免了不必要的复制。

微批次级共享状态管理负责跟踪与并发活跃的流水线微批次相关联的状态，并在前向执行、激活重算与反向传播之间协调它们的生命周期。共享状态会保留到其最终消费者完成计算，随即立刻释放，以限制额外的内存开销。同一运行时抽象还处理阶段布置与源—消费者关系，使注意力实现无需依赖物理流水线布局即可访问共享状态。

只需对优化器、检查点、预热与计算图追踪流程做少量轻量适配，这些机制便能让 CSA2 在既有的分布式训练接口和流水线调度下透明运行。

3.1.3 Engram

Engram 嵌入表按行切分到若干专用进程组上，进程组的大小由 Engram 并行度决定。组的大小控制着单设备内存占用与嵌入查表通信范围之间的权衡。优化器状态进一步在各表分区的多个副本之间分片。Engram 查表的索引只取决于输入词元序列，因此在每个流水线阶段开始处理当前训练步的微批次之前，就会针对整个本地批次发起嵌入预取，从而把对流水线执行的干扰降到最低。反向传播期间，嵌入梯度会被缓存起来，待主干反向传播结束后返还给其所属的 rank。为了与多模态训练高效整合，嵌入预取与梯度传输被安排为与视觉编码器的前向和反向计算重叠。嵌入以 FP8 存储和取回，取回的数值与缩放因子直接传入随后的 GEMM。更新 Engram 表时，Sinkhorn 归一化在整个迭代过程中只维护行、列缩放向量，避免反复写入完整的归一化矩阵；行归一化与部分列统计的累加被融合为单个 kernel，进一步降低内存流量。RL 轨迹采样期间，Engram 嵌入表常驻 GPU 显存。这一布置减轻了主机内存压力，并有助于避免主机内存碎片引发的内存耗尽（OOM）故障。

3.2 推理系统

DeepSeek-V4.1-Flash 在设计上把推理效率作为一等考量。尽管其架构在概念上颇为复杂，最终得到的推理 kernel 流程却出奇地精炼。通过合理的 kernel 融合，我们把纷繁的运算封装进少数几个融合 kernel，并让硬件资源在其中始终保持满流水：包括 FlashMLA（Li and Liu, 2025）中的 fused-RoPE-attention-RoPE-cast kernel，DeepGEMM（Zhao et al., 2025）中的 Mega-Gate、Mega-*m*HC 与 Mega-MoE kernel，TileKernels（Wang et al., 2026a）的 kernel，以及 DeepSelect（Qian et al., 2026）的 TopK kernel。于是，绝大多数 Transformer 层——也就是 CSA2 工作在 Reuse Mode 的那些层——在预填充阶段只需 15 个 kernel、解码阶段只需 11 个 kernel，从而同时实现高吞吐与低时延推理。

在部署层面，我们采用 Encoder-Prefill-Decode（EPD）分离架构，使视觉编码、预填充与解码可以独立扩展，并在执行上相互重叠。

3.2.1 持久 KV 缓存管理

在相同负载下，V4.1 把持久 KV 缓存的占用压缩到 V4 的 1/8。这一缩减来自两个相乘的因子：持久 KV 缓存不再存储 SWA KV，这几乎使其规模减半；保留在其中的全局 KV 又通过架构与精度优化进一步压缩到 V4 占用的 1/4。

在 V4 的部署中，SWA KV 占据了持久 KV 缓存容量的近一半。在该缓存内部，全局 KV 与 SWA KV 各自独立管理，统一采用 LRU 淘汰策略。全局 KV 完整保存，一旦命中即可复用完整前缀。与之不同，SWA KV 只在两个特定位置缓存——prompt 末尾与输出末尾——以便支持重新生成和多轮会话；命中后，计算即可从该缓存位置继续。为了维持高命中率，我们在 SSD 上配置了足够大的持久 KV 缓存，使两类 KV 在典型负载下都能驻留超过 72 小时。尽管只保留了指定位置上的 n_{win} 个 KV 条目，未压缩的 SWA KV 缓存仍带来可观的存储开销，在轮次较短的多轮对话中尤为突出。

把 SWA KV 持久化存储既昂贵又低效，因为其访问模式与持久 KV 缓存的长期保留策略并不匹配。全局 KV 呈现长尾式复用；SWA KV 则只在一个活跃会话内部、以分钟计的狭窄时间窗内被复用，一旦会话结束或下一轮开始便宣告失效。V4 技术报告提出了 *Zero SWA Caching*，通过重算缺失的 SWA KV 来规避存储开销。但精确恢复需要对 $L \times n_{\text{win}}$ 个词元做一次完整前向传播，其代价在生产部署中被证明难以承受。

为此，V4.1 对持久 KV 缓存管理作了如下修改：

1. SWA KV 不再缓存于持久 KV 缓存中，而是存入一个从每台机器主机 DRAM 划出 10% 容量组建的分布式内存池。该池的总容量虽然小得多，但其短 TTL（仅数分钟）能让过期条目立刻回收给新会话使用；在真实负载下，这种高周转足以服务绝大多数并发活跃会话。全局 KV 仍留在持久 KV 缓存中，保底存活时间不少于 72 小时。
2. 驱逐 SWA KV 不可避免会产生未命中，但得益于一个轻量兜底机制——*Encoder SWA Bounded Replay*（详见第 3.2.2 节）——未命中的代价仍可承受。对于那些命中全局 KV 却未命中 SWA KV 的必然但罕见的请求，该机制只需重算 n_{win} 个词元即可恢复缺失状态，而不必对完整的 $L \times n_{\text{win}}$ 个词元做一次前向传播。这一有界重放是整个设计的基石：它把

灾难性的未命中变成平缓而低廉的降级，从而使把 SWA KV 移出持久 KV 缓存的做法站得住脚。

3.2.2 SWA 有界重放

由于 SWA 的依赖关系会随层数不断累积，若要精确重建 L 层的 SWA KV，就需要重放 $L \times n_{\text{win}}$ 个词元。SWA 有界重放则只重放最近的 n_{win} 个词元，并将 SWA 截断至重放片段，接受近似状态：对于从位置 s 开始的重放，位于位置 i 的查询将只关注区间 $[\max(s, i-W+1), i]$ 内的 SWA 键。

编码器 SWA 有界重放。编码器 SWA 有界重放使前缀缓存只依赖全局 KV，从而允许将 SWA KV 从持久化的 KV 缓存中移除。当编码器的 SWA KV 缺失时，我们重放缓存前缀的最后 n_{win} 个词元，并将其与未缓存的后缀一起处理。被重放的词元只重新生成 SWA KV，直接复用缓存的全局 KV，既不重新计算也不覆写；而未缓存的后缀则同时生成全局 KV 与 SWA KV。

按设计，重放得到的前缀状态是近似的，因此为未缓存后缀计算的全局 KV 与 SWA KV 取决于缓存命中位置，在不同位置之间并非数学上完全一致。令人鼓舞的是，我们的实验证据证实，这种有界重放策略几乎不会损害响应质量。

解码器 SWA 有界重放。解码器 SWA 有界重放将解码器的前向传播限制在 n_{win} 个词元以内，使预填充总计算量近乎减半。

在因果编码器—解码器（CED）架构下，解码器的全局 KV 由编码器最终的隐藏状态投影得到。将预填充终止于编码器的唯一障碍是解码器 SWA KV：它由每个解码器层自身的隐藏状态生成，且在最初几个解码步就需要用到。由于我们从不缓存解码器 SWA KV，精确重建它需要在最后 $\frac{L}{2} \times n_{\text{win}}$ 个提示词元上运行解码器各层；当很短的未缓存后缀接在很长的已缓存前缀之后时，这一开销相当可观。因此，我们也把有界重放策略应用到这一场景：每次预填充时，重放提示的最后 n_{win} 个词元，在同样的 SWA 截断下将其编码器输出送入解码器各层，并把得到的解码器 SWA KV 仅用于解码，而不用于前缀缓存。

按设计，重建出的解码器 SWA KV 与完整解码器前向传播的结果在数学上并不等价。此外我们还发现，该策略对响应质量的影响可以忽略不计。为进一步保险，我们在后训练期间也模拟同样的重放，以实现训练感知（train-aware）的适配。

第 4 章 预训练

4.1 数据构建

文本数据筛选。为了追求更高的智能水平，我们不再局限于小规模数据实验所反映的笼统的样本级质量，而是更加关注各类语料之间能够带来独特信息增益的整体相互作用。我们采用更系统、更规范的数据构建流程，以提升数据质量并优化数据配比。具体而言，我们基于更全面的评测，精心设计了针对模型参数量与训练数据量的缩放阶梯，用它来指导大规模训练。我们过滤掉信息增益有限的模型生成内容，包括能力较弱的模型的输出以及低质量的机器翻译文本。我们把这类内容视为隐式重复：它们大多只是对既有信息的重新表述，在长期训练中甚至可能带来负面影响。我们还探索了模型在环（model-in-the-loop）的数据迭代方法，为未来大规模合成数据打下基础。此外，我们邀请更多领域专家参与构建细粒度的数据质量评估维度。与上一版本相比，新语料纳入了更多来自新发布开源仓库、提交记录、代码库与新兴框架的最新代码，覆盖了更广的编程语言范围，也更好地反映了当今真实的软件工程场景。

多模态数据筛选。我们的多模态预训练数据集主要由三类数据构成：图文对、图文交错数据以及领域专用数据。鉴于原始网络数据天然蕴含丰富的多模态知识这一前提，我们没有进行大规模数据合成，而是优先对原生形态的数据进行清洗和利用，以便在预训练中实现最直接、最可扩展的视觉知识压缩。在最初的数据采集阶段，我们发现爬虫系统过度偏向以文本为中心的网络内容，因此基于 Common Crawl 对其重新进行了自举，以改进对多模态来源的覆盖。对于图文数据，我们从网页中连同相关联的替代文本一起提取图像，用图文相关性阈值进行过滤，并按图像语义去重。对于交错数据，该子集主要来自网页和 PDF。处理大规模多模态语料通常比处理纯文本语料产生更高的 CPU 与磁盘开销，这促使我们把交错数据的构建组织成成本逐级递增的多个阶段。在图像检索之前，我们先施加启发式与统计过滤、去重和质量模型，以挑选高价值的文档；幸存的文档随后被组装成图文交错序列，并在序列内再次以图像感知的方式进行过滤和去重；最后，我们使用 SmolVLM（Marafioti et al., 2025）对图文内容进行严格的质量打分，从中提取高质量的交错数据。在此过程中被过滤掉的文档，会经过筛选与重组，部分回收到额外的图文对中。为弥补网络采集数据的固有局限，我们还引入领域专用数据集，以增强模型在细粒度视觉感知（如视觉定位与指向）、光学字符识别（OCR）以及长尾知识获取方面的能力。我们还收集了大量图像—代码对与计算机使用轨迹，以提升多模态智能体理解能力。

数据整合与去重。由于纯文本数据与多模态数据分别经由不同流程处理，我们把最终训练语料构建为两个数据源的并集。对于重叠样本，我们用其多模态版本替换纯文本版本，并采用两种配置中较大的轮数（epoch 数）。经过此次替换，最终语料中纯文本数据与多模态数据的词元比例为 7:1。我们在预训练与上下文扩展期间通过联合预取和分配训练样本，尽量减少样本重叠。超长文档在混合之前先做确定性的预切分，以确保训练词元在各数据分片和各训练步之间均匀分布。我们还进一步改进了最适合（best-fit）打包算法，把填充率控制在至多 10^{-4}。

4.2 预训练设置

4.2.1 模型设置

我们将 Transformer 层数设为 40，隐藏维度 d 设为 5120。我们采用因果编码器—解码器（Causal Encoder-Decoder，CED）架构，编码器为 20 层，解码器为 20 层　解码器为 20 层。前两层使用纯滑动窗口注意力，其余 18 层编码器层使用压缩率为 $m = 2$ 的 CSA2，并划分为三组配置完全相同的 6 层小组。每组的第一层工作于 Full Mode，其余五层工作于 Reuse Mode。20 层解码器层使用压缩率为 $m = 1$ 的 CSA2，划分为五个 4 层小组。第一组中，第一层工作于 Full Mode，其余三层工作于 Reuse Mode；其余四组配置相同，第一层工作于 Reindex Mode，其余三层工作于 Reuse Mode。对于所有 CSA2 层，我们将 indexer 查询头数设为 32，indexer 头维度设为 128，稀疏注意力选取的 KV 条目数（即 attention top-k）设为 512。查询头数设为 64，头维度设为 512，查询压缩维度设为 1280。对于 Hierarchical Sparse Indexer，我们最多选取 2,048 个块、每块 8 个位置，合计最多 16,384 个候选位置。输出投影组数设为 8，每个中间注意力输出的维度设为 1024。在滑动窗口注意力的附加分支中，窗口大小 n_{win} 设为 128。我们在所有 Transformer 模块中采用 MoE 层，并使用带限幅的 SwiGLU 激活函数，限幅阈值为 10。每个 MoE 层由 1 个共享专家和 384 个路由专家组成，每个专家的中间隐藏维度为 2304；对每个词元，将从路由专家中激活 6 个专家。对于 *m*HC，展开因子设为 4，Sinkhorn-Knopp 迭代次数设为 20。视觉编码器方面，层数设为 32，隐藏维度设为 1024，注意力头数设为 16，图像块大小设为 14。视觉 MLP 投影器有 2 层，隐藏维度为 5120。在此配置下，DeepSeek-V4.1-Flash 共有 552B 骨干参数，预填充阶段每词元激活 8B，解码阶段激活 16B。

4.2.2 训练设置

对线性变换的参数，我们采用 Muon 优化器（Jordan et al., 2024; Liu et al., 2025）；对所有 RMSNorm 模块的权重以及其他非矩阵参数，使用 AdamW 优化器（Loshchilov and Hutter, 2019）；对所有嵌入和预测头，则采用 Sinkhorn 平衡更新。对于 AdamW，我们将其超参数设为 $\beta_1 = 0.9$、$\beta_2 = 0.95$、$\varepsilon = 10^{-20}$，权重衰减为 0.1。对于 Muon，动量设为 0.95，权重衰减设为 0.1，并把每次更新矩阵的 RMS 重新缩放到 0.18，以便复用 AdamW 的学习率。对于 Sinkhorn 平衡更新，我们沿用与 Muon 相同的动量系数和学习率校正因子，并设 $K = 11$、$\tau = 10^{-3}$、$\varepsilon = 10^{-20}$。参照（Cheng et al., 2026b）的做法，Engram 的学习率按 5× 缩放。我们在 45T 词元的多模态数据上训练 DeepSeek-V4.1-Flash，全程未出现任何不稳定。整个训练过程批次大小固定为 1.006 亿词元。学习率在前 2000 步内线性预热，随后保持在 2.6×10^{-4} 直到 28T 词元。在 28T 到 40T 词元之间，我们按余弦调度将学习率衰减至 2.6×10^{-5}，并在 40T 到 45T 词元期间保持该值不变。我们从零开始、在序列长度 64K 下带稀疏注意力训练模型，并在 34T 词元处将序列长度扩展到 1M。对于无辅助损失的负载均衡，图像词元和文本词元的偏置更新速度均设为 0.001，同时保留一个权重为 0.0001 的序列级平衡损失，以避免单条序列内部出现极端不均衡。与 DeepSeek-V4 类似，我们在预训练期间采用样本级注意力掩码。

视觉编码器训练。在接入语言骨干之前，DeepSeek-ViT 会经历一个独立的训练阶段。训练流程由两个阶段组成：对比预训练与自回归微调。在对比预训练阶段，我们使用 SigLIP（Zhai et al., 2023）引入的 sigmoid 对比损失，在约 47B 条来自 alt 文本数据的图文对上优化模型。为了从如此规模的海量数据中高效学习视觉表示，我们通过缩小较大图像并保持其长宽比，把最大输入分辨率限制在 224×224 像素。尽管在这一阶段使用更高分辨率能带来可观收益，但实

证结果表明，这些收益对最终模型的贡献很小。由于后续的自回归阶段专门负责高分辨率外推，在对比预训练阶段放大分辨率会显著增加计算开销，而整体提升却很有限。在自回归微调阶段，我们把视觉编码器接入一个 4B 的 MoE 大语言模型，在涵盖图像说明、alt 文本、图表和 OCR 的数据集上，以下一词预测为目标训练 236B 词元。该阶段旨在增强编码器建模细粒度视觉特征的能力。因此，我们把输入分辨率约束在 544×544 至 1344×1344 像素之间，对超界图像按比例缩放。这一阶段结束后，我们舍弃该 LLM，只保留优化后的视觉编码器进入后续预训练流程，并沿用同样的输入分辨率策略。

4.3 评估

4.3.1 评估基准

我们将 DeepSeek-V4.1-Flash-Base 与其前代模型 DeepSeek-V4-Flash-Base 和 DeepSeek-V4-Pro-Base 进行比较。所报告的基准覆盖五个关键维度：世界知识，语言理解与推理，编程与数学，长上下文，以及多模态能力。

世界知识基准包括 AGIEval（Zhong et al., 2023）、MMLU-Pro（Wang et al., 2024b）、C-Eval（Huang et al., 2023）、MultiLoKo（Hupkes and Bogoychev, 2025）、Simple-QA-Verified（Haas et al., 2025）和 SuperGPQA（Du et al., 2025）。

语言理解与推理基准包括 BigBench Hard（BBH）（Suzgun et al., 2022）、BigBench Extra Hard（BBEH）（Kazemi et al., 2025）、DROP（Dua et al., 2019）和 HellaSwag（Zellers et al., 2019）。

编程与数学基准包括 BigCodeBench（Zhuo et al., 2025）、HumanEval（Chen et al., 2021）、GSM8K（Cobbe et al., 2021）、MATH（Hendrycks et al., 2021）和 MGSM（Shi et al., 2023）。

长上下文基准包括 LongBench-V2（Bai et al., 2025）。

多模态基准包括 MMMU-Pro（Yue et al., 2025）、DocVQA（Mathew et al., 2021）、CVBench（Tong et al., 2024）和 RefCOCO/RefCOCO+/RefCOCO-g（Kazemzadeh et al., 2014; Nagaraja et al., 2016; Mao et al., 2016; Yu et al., 2016）。

4.3.2 评估结果

在表 1 中，我们在内部评估框架下、以严格受控且可复现的设置，对 DeepSeek-V4-Flash、DeepSeek-V4-Pro 和 DeepSeek-V4.1-Flash 的基座模型进行了细致比较。与 DeepSeek-V4-Flash-Base 和 DeepSeek-V4-Pro-Base 相比，我们最新的基座模型展现出可观的效率提升：DeepSeek-V4.1-Flash 激活的参数量显著小于 DeepSeek-V4-Pro-Base，KV 缓存占用也大幅降低，而性能与前两者完全持平。这些结果也反映出我们为预训练数据治理流程所做的实质性改进。本版本引入了原生多模态训练，并通过相应的多模态评估验证了其有效性。得益于在更多样、更具多模态特征的语料上训练，DeepSeek-V4.1-Flash 取得了与 DeepSeek-V4-Pro 相当的世界知识和理解能力。在推理与编程基准上，DeepSeek-V4.1-Flash 持续进步，在多个基准上达到接近甚至超过 DeepSeek-V4-Pro 的性能。

为进一步考察模型在真实研发场景中的能力，我们还在专用的内部语料上进行了困惑度测试。由于困惑度测试无法通过模型 API 实施，我们主要关注自己预训练的基座模型。在语料选

表 1 DeepSeek-V4-Flash-Base、DeepSeek-V4-Pro-Base 与 DeepSeek-V4.1-Flash-Base 的比较。所有模型均在我们内部评估框架中测试，并采用相同的评估设置。分差不超过 0.3 的分数视为处于同一水平。每行最高分以粗体显示，次高分以下划线标注。

组别	基准（指标）	样本数	DeepSeek-V4-Flash Base	DeepSeek-V4-Pro Base	DeepSeek-V4.1-Flash Base
架构			MoE	MoE	MoE
激活参数			13B	49B	8B/16B
骨干参数			284B	1.6T	552B
世界知识	AGIEval (EM)	3-5-shot	83.9	**84.4**	83.4
	MMLU-Pro (EM)	5-shot	68.3	73.5	**74.1**
	C-Eval (EM)	5-shot	92.1	**93.1**	92.1
	MultiLoKo (LLM-Judge)	5-shot	42.6	**50.9**	45.5
	Simple-QA-Verified (EM)	25-shot	30.1	**55.2**	42.3
	SuperGPQA (EM)	5-shot	46.5	**53.9**	53.1
语言与推理	BBH (EM)	3-shot	86.9	**87.5**	86.1
	BBEH (EM)	1-shot	25.4	**29.8**	27.2
	DROP (F1)	1-shot	88.6	**88.7**	87.9
	HellaSwag (EM)	0-shot	85.7	**88.0**	87.2
编程与数学	BigCodeBench (Pass@1)	3-shot	56.8	59.2	**60.6**
	HumanEval (Pass@1)	0-shot	69.5	76.8	**79.4**
	GSM8K (EM)	8-shot	90.8	92.6	**93.0**
	MATH (EM)	4-shot	57.4	61.1	**64.5**
	MGSM (EM)	8-shot	**85.7**	84.4	80.2
长上下文	LongBench-V2 (EM)	1-shot	44.7	**51.5**	45.2
多模态	MMMU-Pro (EM)	4-shot	-	-	**56.5**
	CVBench (EM)	4-shot	-	-	**77.9**
	DocVQA (LLM-Judge)	4-shot	-	-	**95.6**
	RefCOCO-avg (Acc@0.5)	0-shot	-	-	**86.0**

择上，我们基于日常研发收集了一套独立的评估集，涵盖内部文档、自有代码仓库和学术资料，面向复杂科学问题与前沿研究的推理、归因与问题求解。结果见图 6，其中我们报告了各模型的每字节比特数（BPB），数值越低表示性能越好。

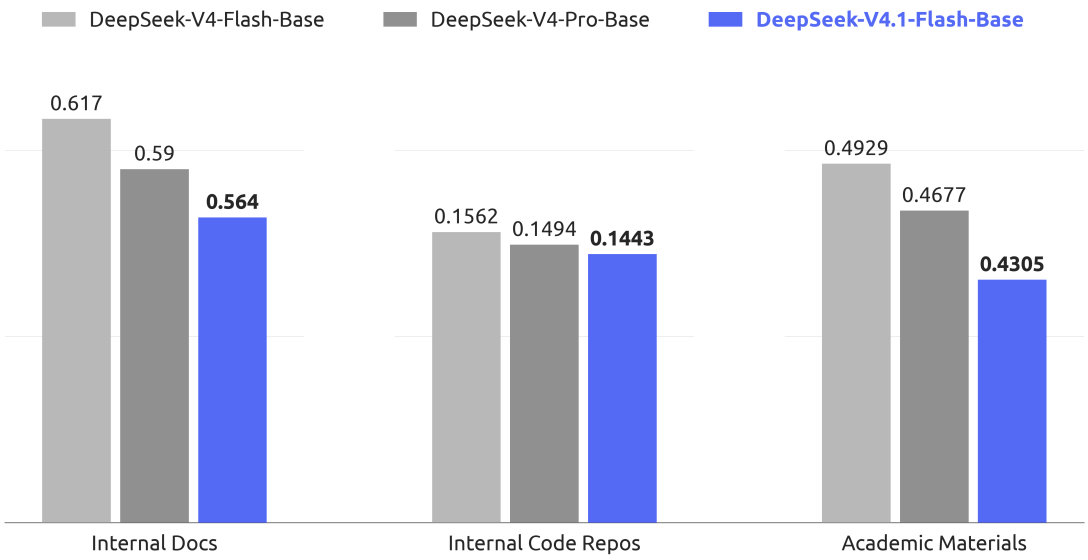


图 6　DeepSeek-V4-Flash-Base、DeepSeek-V4-Pro-Base 与 DeepSeek-V4.1-Flash-Base 在我们留出的评测集上的每字节比特数（BPB）对比。DeepSeek-V4.1-Flash-Base 在所有任务上均取得最低 BPB，展现出成为强大基础模型的更大潜力。

第 5 章 后训练

5.1 后训练流水线

在本发布版本中，我们没有引入新的后训练算法。整体配方遵循监督微调（SFT）之后接强化学习（RL）与在线策略蒸馏（OPD；Gu et al., 2024; Lu and Lab, 2025）的标准范式，除已被充分验证的做法之外，未做任何算法层面的改动。相反，我们的精力几乎全部投向模型训练的对象，而非优化的方式：投入建设大规模、自动化的数据合成与环境构建流水线。具体而言，该流水线（i）合成多样化且可验证的训练任务，并附带参考解与奖励信号；（ii）以程序化方式构建可扩展的交互式 agent 环境，使轨迹能够在其中低成本地采集与评测；（iii）施加严格的过滤、去重与难度校准，以保证数据质量与课程均衡。我们发现，在一个固定且并不出众的优化流程之下，合成数据与环境在规模、多样性和可验证性上的系统性改进，贡献了所观察到的几乎全部收益。这一观察印证了一条更普遍的经验：在现阶段，打磨数据与环境流水线带来的边际收益，显著超过在后训练中追求算法新颖性。

5.1.1 大规模 agent 任务合成

任务是 agent 学习的基本燃料。然而，构建高质量训练任务历来需要大量人工投入。我们观察到，模型已经开始展现出自行构造训练任务的能力，尽管这种能力还远称不上完善。认识到这一潜力之后，我们投入了可观的精力来强化模型的任务构造与质量验证能力。

我们将每个任务形式化为三元组（问题、环境、验证系统），并从两个维度评估其质量：难度，即确保任务并非轻而易举；正确性，即确保三个组成部分中不存在关键缺陷。以难度和正确性作为奖励信号，我们迭代地训练模型去构造更好的任务。我们还对 RL 任务进行全生命周期监控。每当一个任务被用于新一轮 RL 训练，由此产生的轨迹就为质量复审提供了新的证据。在这一总体框架下，我们为两大核心场景——通用 agent 与编码 agent——各建立了一条专门的训练环境生产流水线。

通用 agent。对于通用 agent，我们鼓励内部员工与外部合作伙伴将最新模型纳入日常工作流程，并在自愿的基础上回传交互数据与反馈。依据回传数据中观察到的接口，我们构造了一套大规模的模拟工具，用以复刻真实工具与系统的接口和行为，涵盖输入格式、输出结构、APIschema 以及行为约束，既包括常用的 SaaS 与企业应用，也包括更专门的业务后端系统。与此同时，我们大规模收集内部员工提交的负反馈与模型失败案例，并把它们汇入流水线，生成扎根于真实工作流的单轮与多轮 agent 环境。通过重建相关的工具上下文、用户交互模式和失败条件，该流水线能够系统性地重放失败情形，并针对观察到的模型弱点开展有针对性的强化学习。

编码 agent。编码 agent 训练环境来自两个来源：1）内部员工与外部合作伙伴的编码 agent 会话，经筛选后只保留高度复杂的任务或模型表现不佳的任务，再按轨迹去重；2）星标数达到设定阈值的公开 GitHub 仓库。环境构建由多个专门的 agent 协同完成。首先，一个 agent 判断该项目能否在容器内完成构建并完整运行、能否被自动验证；若能，它会选取某个特定的轮次或提交（commit）作为任务起点，设计若干复杂度足够的实现方向，并给出具体的评测点（同时包含 fail-to-pass 与 pass-to-pass 评测点）以及一份构建报告，必要时从网络获取外部资源。随后，

另一个 agent 在隔离容器中配置依赖、初始工作目录、测试代码与任务描述，执行自测，清除一切可能泄露任务解法的痕迹，并把环境打包成一个新的镜像层。接着，多个互不相同的 agent 尝试解决该任务，由一个独立的质量检查 agent 连同各解题 agent 的轨迹审查该环境，检查环境问题、事实性错误、评测点与任务描述之间的不一致，以及环境可被投机取巧（hackability）的风险。若检查未通过，修复 agent 会修复全部已发现的问题，调整过易或过难的评测点，然后任务重新进入验证环节。

借助这些流水线，我们能够批量自动产出正确、有区分度且长度与难度可控的 RL 训练数据。无论是通用 agent 环境对真实工作流的忠实重建，还是编码 agent 环境对编码任务的精确构造，最终都汇入同一套训练系统，在持续的质量监控下推动模型能力的迭代提升。

5.1.2 在合成任务上开展 RL

我们利用大规模异步 RL 在合成任务上提升模型性能，并塑造其在复杂场景中的行为。我们在两个维度上扩展 RL 训练的规模，即训练算力与脚手架（scaffold）的数量。如图 7 与图 8 所示，随着累计 RL 步数带来的算力增加，性能持续提升，无论是单个脚手架内部的扩展、同一脚手架各变体之间的联合扩展，还是跨异构脚手架的扩展，均是如此。

对于跨多种不同脚手架的 RL 训练，我们把 agent 轨迹采样的执行解耦为 agent 沙箱与工作容器。沙箱运行脚手架及其工具，而工作容器提供一个与脚手架无关的控制层，负责编排轨迹采样，把异构的交互规整成统一的轨迹模式（schema），并与训练器通信。

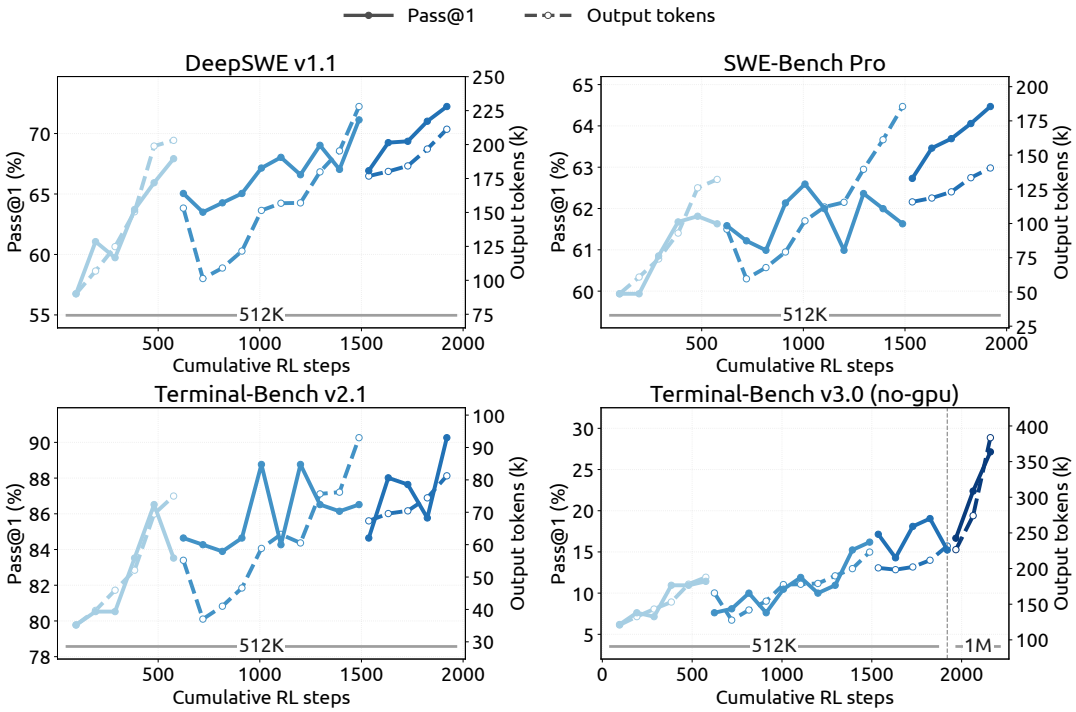


图 7　在 DeepSeek Harness 的 Minimal 模式下，随着强化学习训练规模的扩大，模型在多个代码智能体基准上的性能持续提升；把最大上下文长度进一步扩展到 1M 词元后，在极长时程任务（例如 Terminal-Bench v3.0）上的性能仍继续改善。

……二者都运行在 DSec（第 5.1.3 节）上，位于可抢占 GPU 训练池之外，从而把长周期运行的轨迹采样与细粒度的训练调度分离开来。训练器被抢占期间，轨迹采样的执行可以在保持

完整状态的前提下挂起并卸出（offload），以便之后恢复，同时释放 CPU 与 GPU 资源。这一设计无需修改底层算法，就能在异构脚手架之间实现稳定而高效的强化学习。

为了把有效的强化学习算力拓展到单次训练运行之外，我们用模型合并（model merging）来重新初始化后续的强化学习运行。具体做法是，合并来自不同脚手架或不同配置的运行得到的检查点，把沿不同优化路径积累的改进融合起来。图 7 和图 8 中的非连续曲线段，反映的正是模型重新初始化之后相继进行的强化学习运行。这一做法在任务性能和 token 效率上都带来进一步提升，为聚合并行的强化学习算力、并在一次又一次运行之间持续扩展规模提供了一种简单实用的途径。

5.1.3 大规模运行智能体：DSec

在从 DeepSeek-V3 迈向 V4 的过程中，智能体训练环境的数量与多样性迅速增长，促使我们建设了 DeepSeek Elastic Compute（DSec），一个面向大规模智能体训练与评估的生产级沙箱平台。其初始设计解决的是异构执行环境、可扩展的镜像分发、多种隔离后端、高密度资源管理、命令轨迹日志，以及抢占后安全恢复等问题。

V4.1 的训练进一步把需求推高到数百万个并发沙箱实例，涵盖多样的脚手架（harness）、平台、代码仓库、软件依赖和任务专用服务。在这个规模上，主要瓶颈转向了数据中心可扩展性、负载隔离、单节点计算密度，以及如何遏制能力日益增强的智能体出现不当行为。下面我们简要介绍几项关键设计。

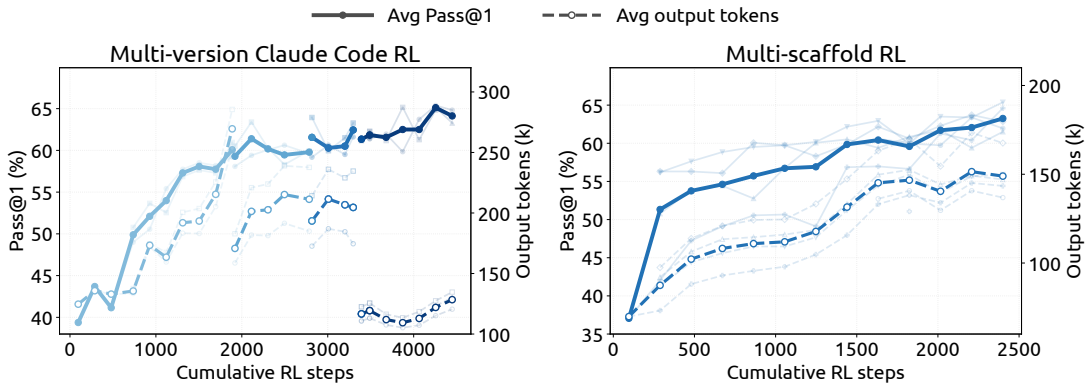


图 8　跨 Claude Code 的多个版本联合训练（左图），以及跨异构脚手架——包括 OpenCode、Pi 以及处于 Standard 和 PTC 模式的 DeepSeek Harness——联合训练（右图）时，性能随累计强化学习步数提升。评测在 DeepSWE v1.1 上进行。较浅的曲线为单个脚手架版本或单个脚手架各自的评测结果。

计算的大规模横向扩展。DSec 依靠两种互补机制来扩展：分片（sharding），以及一致性宽松（relaxed consistency）的调度。为了容纳大量机器，我们把计算节点划分成多个分片（即所谓的规模单元，scale unit）。这种分片还能隔离不同实验的负载、从而缩小故障影响范围（blast radius），避免单个内存消耗大的任务耗尽供其它无关工作共享的资源。

DSec 没有使用 Kubernetes 这类现成的编排器，而是采用一个定制的放置引擎来调度如此大量的沙箱，以牺牲强全局一致性换取可扩展性。这一设计基于一个关键观察：只要每个计算节点都强制执行本地安全约束，智能体沙箱的放置就只需要最终一致性。为此，放置引擎部署成多个相互独立的副本，彼此不做同步协调。每个副本根据近期测量预测资源可用性，并做出

“足够好”的放置决策。为了弥补一致性上的损失，每个节点负责校验最终的放置决策，并执行一条硬性准入约束：一旦超过本地告警阈值，就拒绝新的放置。整体上，这一设计让 DSec 能够扩展到数百万个容器，而不会因中央协调形成瓶颈。

沙箱的高密度运行。在节点层面，我们使用硬件支持的子 NUMA 划分（sub-NUMA partitioning），并把每个工作虚拟机（VM）绑定到单独的 NUMA 域。容器运行在这些工作虚拟机之内，其 CPU 与内存分配被限制在该虚拟机的本地 NUMA 资源内。这种配置在激进的内存超售与 Linux 内核锁争用之间取得平衡，同时把内存压力和运行期故障局部化。在可类比的负载配置下，它在端到端性能出现可测量退化之前，把每台物理节点可支撑的并发存活容器数量从约 1,000 提升到 2,500 以上。

不过，这种高密度部署仍可能因后台负载的干扰而影响对时延敏感的评估的准确性。因此，DSec 引入了时延敏感（latency-sensitive，LS）执行类别。我们对非 LS 任务应用 `SCHED_IDLE`，把它们的调度优先级压到最低，

并利用核心调度，确保只有属于同一优先级类别的任务才会同时执行于兄弟超线程（sibling hyperthread）之上，从而消除相互干扰。

不当行为智能体的处置。在 RL 训练过程中，我们经常观察到智能体（agent）试图进行奖励作弊（reward hacking），或无意中把运行环境弄崩溃。在某些尝试中，我们的智能体利用了近期披露的种种漏洞，包括 XFS 驱动中的权限问题、AppArmor 中的非法内存访问、从软件包镜像服务中泄露答案，等等。智能体在删除关键二进制文件、破坏系统文件、甚至把整个文件系统一并删掉方面也向来“臭名昭著”。我们为每个沙箱单独配置 AppArmor 策略，并辅以基于 eBPF 的细粒度网络策略来阻止此类尝试。如果某个智能体把自己的环境弄崩溃，我们就把这次崩溃记为一条失败的轨迹，并向 RL 框架上报一个“repercussion”（后果反馈）信号。

5.1.4 RL 中可控的推理强度

在模型架构与硬件不断进步的同时，输出 token 的数量是服务成本的另一个关键决定因素，进而也决定了真实应用中的成本—质量权衡。因此，我们在强化学习过程中引入一个标量的推理强度级别 b，将其作为显式的条件信号。该机制同时适用于单轮推理与多轮智能体任务。具体而言，我们在系统提示词（system prompt）开头加入如下指令：

`Reasoning Effort:{effort}`（取值范围 1–100；数值越高，所要求的推理越彻底）。

其中，$b \in \{1, \ldots, 100\}$ 表示所请求的推理强度级别。

对于每个训练提示词 x，我们在每个强度级别 $b \in \mathcal{B}$ 上采样 M_b 条响应：

$$z_{b,j} \sim \pi_\theta(\cdot \mid x, b), \qquad b \in \mathcal{B}, \quad j = 1, \ldots, M_b. \tag{8}$$

其中，j 为在强度级别 b 下所采样各条响应的编号。共享同一 (x, b) 的响应构成一个子组（subgroup），组内奖励先做均值中心化，再用于计算组内相对优势（group-relative advantage）。因此，来自不同强度级别的响应不会被直接比较；恰恰相反，强度相关的行为是在每个子组内部诱导出来的：奖励中的长度分量取为

$$r^{\text{len}}_{b,j} = -\min\left\{C_{\max},\ k(b)\frac{\ell_{b,j}}{L_{\text{norm}}}\right\}, \tag{9}$$

其中，$\ell_{b,j}$ 为推理 token 的数量，L_{norm} 为参考长度，$C_{\max}$ 为对单条轨迹所能施加扣分的上限。token 惩罚系数随强度级别呈指数衰减：

$$k(b) = k_0 \exp\left(-\frac{b - b_{\min}}{\tau}\right), \qquad \tau = \lambda \overline{\Delta b}, \tag{10}$$

其中，k_0 为各强度级别共用的基础惩罚系数，$b_{\min}$ 为 $\mathcal{B}$ 中的最小值，$\overline{\Delta b}$ 为相邻训练强度级别之间的平均间隔，λ 控制惩罚衰减的速率。b 每增大 τ，惩罚系数就乘以 e^{-1}。参数 k_0 控制促使推理变短的总体压力大小，而 τ 越小，惩罚衰减得越快，越倾向于在强度级别之间拉开更大的行为差异。附录 C 为 $k(b)$ 的指数参数化提供了基于边际效用的动机。

在部署阶段，标量 b 为模型推理强度提供了一个灵活的控制接口。通过改变 b，同一个模型检查点就能沿学到的成本—质量前沿在不同运行模式之间切换，以适应不同的延迟、token 预算和解题质量要求。尽管训练只使用了有限的强度级别集合，部署时仍可采用中间取值来诱发介于其间的推理行为，为测试时资源分配提供一种细粒度且高效的机制。

在我们于 2026 年 9 月上线的生产部署中，公共 API 暴露出三个预设的推理强度档位——max、high 和 low——它们直接映射到这一标量接口。如表 2 所总结，三个档位分别对应强度值 $b = 100$、$b = 75$ 与 $b = 50$；这样一来，API 用户无需改动模型权重或解码配置，即可在学到的成本—质量前沿上选定一个运行点。

表 2　公共 API 推理强度档位与底层标量强度值 b 的对应关系

API 档位	强度值 b
max	100
high	75
low	50

5.2 异步后训练基础设施

在面向 LLM 的 RL 训练中，轨迹采样阶段的长尾问题一直是拖累训练效率的主要瓶颈。为解决这一问题，我们把后训练基础设施扩展为支持样本的异步生成（Zeng et al., 2026; Team et al., 2026b），通过维持足够高的并发水平，显著缓解了轨迹采样阶段的长尾问题。目前，我们几乎所有 RL 与 OPD 任务都已启用异步训练，轨迹采样效率也得到了大幅提升。

5.2.1 整体流程

我们把轨迹采样（rollout）与训练共置于同一批物理设备上，并对二者的执行分时共享，从而不再需要在两个阶段之间人工调优资源分配。每个任务都会为同时在途（in-flight）样本的数量指定一个上限，系统则在整条轨迹采样过程中始终维持这一上限。

为了维持目标的轨迹采样并发度，我们评估了三种派发（dispatch）粒度。我们的最终方案是样本级派发：一旦新完成样本的数量达到分配给下一提示词的 GRPO 组大小，无论这些补全（completion）出自哪些组，都立即派发该提示词。这有助于在整个训练过程中保持稳定的轨迹采样并发度。在确定这一派发策略之前，我们还尝试过另外两种派发粒度。第一种尝试是在训练开始时先多派发若干批次，并在每次训练迭代后补充一个完整批次；然而，这会造成训练指

标剧烈震荡，说明批次级粒度过粗。第二种尝试改为提示词级派发，即每当一个 GRPO 组完成就派发一个新的提示词，但发现它很容易在 GRPO 组内的长尾样本上停滞，难以平稳地维持目标轨迹采样并发度。

一旦积累起足够多的训练样本，训练就会抢占仍在进行的轨迹采样。 训练中，我们采用级联路由重放（concatenated routing-replay）：对于跨越多个检查点的样本，我们把每个轨迹采样段由当时检查点产生的专家路由拼接起来，而不是丢弃之后再改用新检查点重新计算路由信息。

5.2.2 缓解长度偏差与异策略效应

异步生成在切实提升轨迹采样效率的同时，也会引入两个可能损害训练质量的副作用。其一，它会带来长度分布偏差，在训练早期尤为明显：较短的序列通常先完成，因而主导了最初的训练批次。其二，它不可避免地产生异策略（off-policy）样本，即其词元部分或全部由较早检查点生成的样本。这两个问题需要不同的应对策略。

为解决长度偏差，我们采用两个机制。其一，调度器可以按数据集限制并发量，这有助于调节各数据集在稳态训练批次中的占比，通过控制进入样本的来源间接缓解长度偏斜。其二，我们支持丢弃提前返回的短样本，以平滑过渡到稳态长度分布，并防止模型过拟合于过短的序列。

针对异策略问题，我们实现了另外两个机制。其一，通过调整控制样本分发与训练样本等待条件的逻辑，我们可以限定最大异策略比例，确保训练数据不会过度偏离当前模型。其二，训练期间加入损失掩码方案，剔除陈旧程度过高的词元的贡献，从而减轻陈旧样本对梯度更新的不利影响。

5.2.3 性能优化

在我们的异步强化学习框架中，轨迹采样阶段会被周期性打断，以便切换到更新之后的策略检查点。我们力求让这一过程无缝进行：中断应近乎瞬时，被打断的轨迹采样应恢复得如同从未停下。

为及时终止轨迹采样，我们支持词元级中断：生成可以在任意词元边界处停下。一旦收集到足够的训练数据，所有在途样本几乎立即停止，系统得以无延迟地进入训练阶段。

为了在检查点切换之间保留轨迹采样的进度，KV 缓存、专家路由等采样状态在生成期间以词元粒度持久化保存。改用新检查点恢复时，已持久化的状态被直接复用，免去了重新预填充的开销，被打断的样本得以从断点处精确续跑。由于这要求保留所有在途样本的状态，我们执行样本粒度的垃圾回收，每个样本一经完成就立即释放其状态。

除了检查点切换之外，同一套快速中断、无缝恢复的机制还让训练任务能够及时响应集群调度的抢占信号而不丢失进度，从而提升集群的整体利用率。

5.2.4 大规模同策略蒸馏

作为后训练的最后一个阶段，最终的全词表 OPD（on-policy distillation，同策略蒸馏）任务在汇集所有领域的数据集上训练，使用超过 40 个教师模型。该任务同样采用异步生成来提高轨

迹采样效率。由于各领域训练流程存在差异，每个领域的最佳教师可能来自模型开发的不同阶段；此外，各教师模型彼此之间在架构上可能互不相同，与学生模型的架构也可能不同。我们的后训练基础设施能够轻松适应这一设定：支持架构异构、数量实际上不受限制的教师进行全词表 OPD，并在它们之间以近乎可忽略的代价高效切换（DeepSeek-AI, 2026b）。

OPD 阶段在训练期间同样要求动态重配置。我们持续跟踪模型能力，并可能随之调整训练配方，包括数据集配比、每数据集并发上限和启用的教师。在同步训练中，这类调整很直接，因为轨迹采样批次提供了显式的配置边界；而在异步场景下，不同配置下生成的样本可能在途中并存。我们的基础设施支持配置之间的一致切换，且不打断轨迹采样或训练。

5.3 评估

5.3.1 评估设置

我们的后训练评测主要聚焦于推理能力与智能体能力；知识密集型表现主要由预训练决定，报告见表 1。推理方面，我们在 GPQA Diamond（Rein et al., 2023）、Humanity's Last Exam（Phan et al., 2025）、Codeforces（内部基准测试）以及 MathArena Apex（Dekoninck et al., 2025）上进行评测，温度与 top-p 均设为 1.0。智能体能力方面，我们分四个类别进行评测：

- **代码智能体**：Terminal-Bench 2.1（Merrill et al., 2026）、Terminal-Bench 3.0（Marten et al., 2026b）、Terminal-Bench 4.0（Marten et al., 2026a）、DeepSWE v1.1（DataCurve, 2026）、ProgramBench（Yang et al., 2026）、NL2Repo-Bench（Ding et al., 2025）。
- **网络安全**：SEC-Bench Pro 版本 260505（Lee et al., 2026）、CyberGym（Wang et al., 2026c）与 ExploitGym（Wang et al., 2026b）。
- **通用智能体**：AutomationBench v1.0.6 的公开评测集（Shepard and Salimans, 2026）、Agents' Last Exam（Sun et al., 2026a）（ALE-CLI）。
- **视觉智能体**：Chartography（Garre et al., 2026）、BabyVision（Chen et al., 2026）、ZeroBench 主集（Roberts et al., 2025）。

对于代码智能体，我们使用 DeepSeek Harness 的 Minimal 模式评测 DeepSeek-V4.1-Flash，上下文窗口为 1M-token，温度设为 1.0，top-p 设为 0.95。为对齐官方设定要求，DeepSWE v1.1 采用 mini-SWE harness。对于 SEC-Bench Pro，我们专门使用 Claude Code harness，取其会话压缩设计。对于视觉智能体任务，我们使用 Claude Code harness 评测，上下文窗口为 512k-token，温度设为 1.0，top-p 设为 0.95。Agents' Last Exam 与 AutomationBench 使用各自的官方脚手架评测。模型在其他编码脚手架下的性能报告于表 4。

为在代码智能体评测中抑制奖励投机（reward hacking），我们限制互联网访问，并从环境中移除 Git 历史记录。此外，我们会在各类环境中自动清理临时构建与包缓存，包括 Go 模块缓存（`go/mod`）、`node_modules` 依赖产物、编译出的 `.jar` 文件以及 Python 的 `__pycache__` 目录。尽管采取了这些防范措施，测试期间我们仍观察到寻找漏洞利用的行为——例如在 CyberGym 中反编译 Ubuntu Linux 核心软件包来挖掘漏洞。随着模型能力日益增强，标准评测基础设施（例如 Docker 容器 与验证脚本）也更容易被模型投机利用。我们呼吁更广泛的研究界在设计下一代基准（测试）时，把检测并抑制这类行为放在优先位置。

5.3.2 评测结果

表 3 DeepSeek-V4.1-Flash 与闭源/开源模型的对比。† 表示 HLE 的纯文本子集。最佳结果加粗，次佳结果加下划线。

基准（指标）	Opus-5 Max	GPT-5.6 Sol Max	K3 Max	GLM-5.3 Max	DS-V4-Pro Max	DS-V4-Flash Max	DS-V4.1-Flash Max
Reasoning							
GPQA Diamond（Pass@1）	**93.4**	92.9	88.1	-	92.4	89.9	90.9
HLE（Pass@1）	42.0†	42.7†	37.8†	39.1†	**56.3**	44.5	43.5
Codeforces（Rating）	-	-	-	-	3348	3289	**3471**
MathArena Apex（Pass@1）	-	-	**65.6**	-	65.3	58.6	**65.6**
Agentic							
Terminal-Bench 2.1（Pass@1）	89.1	88.8	88.3	88.2	87.9	82.7	**90.6**
Terminal-Bench 3.0（Pass@1）	**43.3**	34.4	17.7	28.3	11.8	7.6	30.0
Terminal-Bench 4.0（Pass@1）	**51.8**	39.9	12.6	37.9	12.4	7.0	31.2
DeepSWE v1.1（Resolved）	74.0	73.0	67.5	66.9	62.7	54.4	**74.2**
ProgramBench（Almost@1）	**37.0**	23.0	17.5	19.0	15.5	-	20.3
NL2Repo-Bench（Score）	**75.3**	56.8	58.0	58.0	61.5	54.2	65.4
CyberGym（Pass@1）	-	84.5	80.0	84.5	83.3	76.7	**88.1**
SEC-Bench Pro（Pass@1）	-	**74.3**	-	-	56.4	30.9	62.8
ExploitGym（Pass@1）	22.1	**33.7**	-	15.0	5.4	1.8	15.3
HLE w/ tools（Pass@1）	63.6	-	59.8	62.5	60.0	51.5	**63.9**
Automation-Bench（Pass@1）	50.3	45.8	46.7	48.8	43.2	37.7	**54.8**
Agents' Last Exam（Pass@1）	28.6	26.7	27.6	28.5	25.7	25.2	**31.8**
Chartography w/ tools（Pass@1）	**84.0**	79.9	68.1	-	-	-	78.9
BabyVision w/ tools（Pass@1）	**94.1**	88.9	85.7	-	-	-	89.6
ZeroBench-main w/ tools（Pass@5）	52.0	**53.0**	41.0	-	-	-	49.0

如表 3 所示，DeepSeek-V4.1-Flash 相较其前身 DeepSeek-V4-Flash，在推理基准与智能体基准上均有显著提升，同时达到乃至超过一流开源模型与专有模型的水平。

在核心推理任务上，DeepSeek-V4.1-Flash 在 Codeforces 上取得 3471 的评分，高于 DeepSeek-V4-Flash（3289）与 DeepSeek-V4-Pro（3348）。在 MathArena Apex 上，其 Pass@1 准确率为 65.6%，与表现最好的开源基线 Kimi-K3（65.6%）完全持平，并与 DeepSeek-V4-Pro（65.3%）相当。此外，其 GPQA Diamond 得分达到 90.9%，较 DeepSeek-V4-Flash（89.9%）稳步提升。

在智能体任务上，性能提升更为显著。在 DeepSWE v1.1 上，DeepSeek-V4.1-Flash 通过率达到 74.2%，较 DeepSeek-V4-Flash（54.4%）大幅跃升，并超过 Opus-5（74.0%）与 GPT-5.6 Sol（73.0%）等一线专有模型。在 Terminal-Bench 2.1 上，它取得 90.6%，高于 Opus-5（89.1%）与 GLM-5.3（88.2%）。同样，在 Automation-Bench（54.8%）和 Agents' Last Exam（31.8%）上，DeepSeek-V4.1-Flash 相对同类开源模型与顶级闭源系统均确立了领先成绩。尽管体型紧凑，DeepSeek-V4.1-Flash 仍展现出最先进的智能体能力：在大幅缩小与前沿闭源模型差距的同时，在开源模型中建立了明显优势。

在网络安全任务上，DeepSeek-V4.1-Flash 在开源模型中确立了新的最优水平。鉴于这类能力具有双重用途属性，我们鼓励社区负责任地使用它们，例如用于防御性安全研究与漏洞修复。

在视觉智能体任务领域，DeepSeek-V4.1-Flash 展现出稳健的能力，尤其是在需要视觉推理以及分析复杂专业图表的场景中。虽然它的表现优于领先的开源模型 Kimi-K3，我们也承认，与领先的闭源模型相比仍存在可测量的差距。

除峰值性能之外，DeepSeek-V4.1-Flash 还提供推理强度设置，让用户能以可控的方式在推理成本与准确率之间做权衡。如图 9 所示，随着强度取值提高，推理基准与智能体基准上的准

确率和输出长度都在稳步上升。把强度从 25 提高到 100，八个推理密集型基准的平均 Pass@1 从 67.1% 提升到 76.3%，DeepSWE v1.1 从 66.0% 提升到 74.2%，Terminal-Bench 2.1 从 82.4% 提升到 90.6%，代价是输出词元数量约增加 2.5×。在单响应推理上学到的强度控制，能够忠实地迁移到长程智能体轨迹中，并在多轮过程中控制探索与验证的总量。增益集中在前期：强度处于 60–80 区间时，用不到最大档一半的词元预算就能恢复最大档的大部分准确率；而迈向强度 100 的最后一步会让智能体轨迹延长 1.6–1.8×，换来的提升却微乎其微。因此，最高档最好留给最具挑战性的任务；就日常智能体使用而言，中等强度档位能提供更有利的性价比。

5.3.3 不同推理强度下的性能

图 9 报告了不同预算值下的性能与平均回复长度。随着推理强度增大，模型的推理轨迹逐渐变长，在推理密集型基准与软件工程任务上的准确率单调提升；收益主要集中在中低预算区间，超过该区间后边际收益递减。虽然我们的 RL 训练只涉及为数不多的几个强度档位，但标量强度仍能在特定区间内对回复长度实现灵活的插值控制。这使使用者得以沿一条平滑的连续区间，在质量与延迟、词元成本之间做折中：对延迟敏感的应用可以用低强度运行，准确率只有轻度下降；而困难任务可以调用高强度，恢复模型完整的推理能力。在我们的公开 API 服务中，我们提供映射到该尺度的三个预设强度档位：low、high 和 max 分别对应强度值 50、75、100。

5.3.4 不同智能体脚手架下的性能

实际部署中，模型很少固定在一个单一不变的智能体框架里运行；不同脚手架的系统提示词、工具定义、上下文管理策略与交互协议各不相同，过拟合于某个特定框架的模型一旦换到另一个框架，性能可能大幅下滑。为评估我们的模型对这类差异的鲁棒性，我们的对比覆盖六个脚手架家族的八种配置：Claude Code（Anthropic, 2026）、Codex（OpenAI, 2026）、OpenCode（Anomaly, 2026）、Pi（Zechner, 2026）、mini-SWE（Yang et al., 2024），以及提供 Minimal、Standard、PTC 三种模式的 DeepSeek Harness（DSH）（DeepSeek-AI, 2026a）。对每种脚手架，我们都保持模型检查点、解码配置与任务集不变，仅更换外围框架，包括其原生系统提示词、工具定义与对话轮转逻辑。表 4 报告了 Max 推理强度（100）下 DeepSWE v1.1 与 Terminal-Bench v2.1 的结果。

该模型的智能体能力能够良好地迁移到使用不同提示词与工具接口的各个脚手架家族，而不依赖某个特定框架的专门约定。随着外围交互协议与工具抽象的变化，其性能保持稳定，说明其智能体行为并未与某一个单一脚手架设计紧耦合。这种鲁棒性与我们合成训练数据（第 5 节）中环境、工具定义与交互格式的多样性相吻合，而这些数据在设计上就是为了促进跨智能体脚手架的泛化。

表 4　Max 推理强度下不同智能体脚手架的性能。

基准（指标）	Claude Code	Codex	OpenCode	Pi	mini-SWE	DeepSeek Harness		
						Minimal	Standard	PTC
DeepSWE v1.1（Resolved）	69.8	65.6	65.5	66.2	74.2	72.6	70.5	67.6
Terminal-Bench v2.1（Pass@1）	88.0	84.1	85.0	86.1	90.3	90.6	85.8	85.8

注：所有脚手架都在 DeepSWE v1.1 上对每个任务采样 8 次，在 Terminal-Bench v2.1 上采样 3 次。运行均在 Linux 容器中进行，温度 1.0，top-p 0.95，上下文窗口 1M 词元，并在两个基准上都将每个智能体的模型生成轮数上限设为 max_steps=500。Terminal-Bench v2.1 的评测在无网络访问条件下进行。我们评测了四个版本的 Claude Code，各版本结果及其平均值见附录表 5；本表报告 v2.1.251。更多与脚手架相关的配置详见附录 B.1。

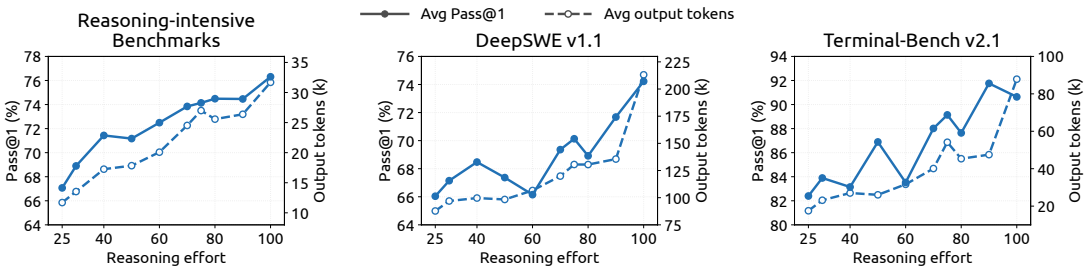


图 9　性能与输出长度随推理强度的变化。各面板绘制 Pass@1（实线，左轴）与每条回复的平均输出词元数（虚线，右轴），推理强度取值从 25 变化到 100；推理密集型基准的结果为八个基准（AIME 2026、Apex 2025 Shortlist、GPQA Diamond、HLE、IMO-AnswerBench、LiveCodeBench、MathArena-Apex、SimpleQA-Verified）的平均值。DeepSWE v1.1 基于 mini-SWE 评测，Terminal-Bench v2.1 基于 DeepSeek Harness（Minimal）评测。

5.3.5 多智能体

为探索复杂任务上的多智能体协作，我们使用 DeepSeek Harness 的 Agent Team 模式对 DeepSeek-V4.1-Flash 进行了初步实验。

多智能体框架。我们在 Agent Team 模式下使用 DeepSeek Harness：默认情况下，主控智能体可以通过 `spawn_teammate` 异步创建具名且持久的队友。每个队友接收一个被委派的任务，并以两种方式之一启动：fresh 模式（不带主控的历史记录），或 fork 模式（附带主控已完成回合的一次性快照）。所有智能体共享同一个仓库检出，因此各方的修改彼此即时可见。

各智能体通过一个持久的对等信箱进行通信。经 `send_message` 发出的消息，会在运行中的队友的下一个步骤边界送达，会为空闲的队友开启新回合，或唤醒不活跃的队友。在每个队友的任务生命周期内，主控智能体使用 `list_agents` 监控运行状态，并使用 `wait_agent` 等待状态、信箱或共享任务的变化。任务归属、依赖关系和建议性写入范围维护在一个共享任务看板上（使用四个 `team_task_*` 工具，更新时进行版本核查）。

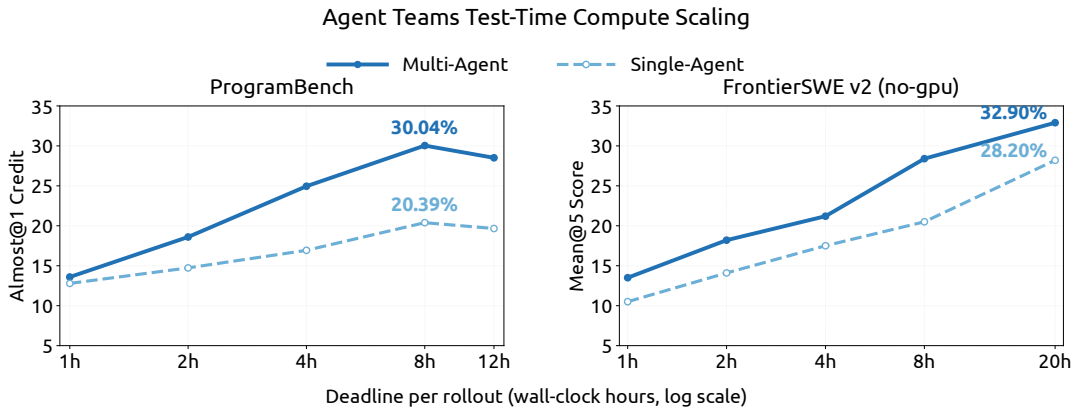


图 10　单智能体与多智能体配置在 ProgramBench（Almost@1）与 FrontierSWE v2（Mean@5）上的测试时计算量扩展，横轴为单次轨迹采样的墙钟期限。

需要介入时，只有领队（lead）可以通过 `interrupt_agent` 打断队友智能体当前正在进行的回合。所需工作完成后，由领队审查并测试汇总后的全部改动，并产出最终回复。

训练。我们用强化学习奖励训练 Agent Team 模式，该奖励由三部分组合而成：任务性能、鼓励任务委托与智能体间通信的协作加分，以及促进高效协调的派生时延惩罚。派生时延的计

算方法是：把执行事件及其协作依赖表示成有向无环图（DAG），按固定的预填充/解码速率由词元数计各事件的成本，再加上实测的工具执行时间，然后取关键路径的长度。这样既鼓励有用的并行，又惩罚不必要的串行工作与同步，同时降低了对服务端批处理与排队时延的敏感度。

性能。我们从 ProgramBench（Yang et al., 2026）构建一个高置信度子集，只保留参考解在隐藏测试集上通过率不低于 95% 的任务，筛选后剩下 172 个“golden”任务。我们还在 FrontierSWE v2（Kondra et al., 2026）上评估，它是 FrontierSWE 更大、更具挑战性的后继版本，并采用了大幅改进的评测方法。我们基于当前公开的任务，排除需要 GPU 的任务，构建了一个无 GPU 子集。我们在两个基准上、在显式给定的单次轨迹采样墙钟期限（wall-clock deadline）下评估单智能体与多智能体配置。此处报告的是初步结果：我们把观测到的最强多智能体配置与可用的最强单智能体基线作比较。在 ProgramBench 上，每个任务最多运行三次轨迹采样（rollout），对应每种配置 516 次计划内的轨迹采样。我们报告 Almost@1，它衡量单次轨迹采样得分不低于 0.95 所占的比例。在 FrontierSWE v2 上，我们报告 Mean@5。ProgramBench 的期限范围为 1 至 12 小时，FrontierSWE v2 则在 1 至 20 小时的期限下评估。在每个期限处，指标都根据期限到达时可获得的输出计算。

如图 10 所示，在两个基准的每个期限下，多智能体配置的表现都优于对应的单智能体配置。在 ProgramBench 上，多智能体配置的 Almost@1 从 1 小时的 13.59% 升至 8 小时 30.04% 的峰值，而单智能体配置分别为 12.79% 与 20.39%。在 FrontierSWE v2 上，多智能体配置的 Mean@5 从 1 小时的 13.50% 升至 20 小时的 32.90%，而单智能体配置从 10.50% 升至 28.20%。

第 6 章 结论、局限性与未来方向

本文提出 DeepSeek-V4.1-Flash，一个支持最长一百万词元上下文的多模态混合专家（Mixture-of-Experts）模型。通过对模型架构、缓存精度与部署策略的联合优化，DeepSeek-V4.1-Flash 把 KV 缓存压缩推向新的极限。其因果编码器-解码器（Causal Encoder-Decoder, CED）架构使模型在预填充阶段每个词元只需激活 8B 参数，而解码阶段为 16B，从而提升了输入密集型智能体工作负载的成本效率。在相同序列长度下，压缩稀疏注意力 2（Compressed Sparse Attention 2, CSA2）中的跨层 KV 缓存复用与 FP4 KV 缓存把其全局 KV 缓存占用（始终位于 HBM 中）降至每词元 890 字节，约为 DeepSeek-V4-Flash 对应占用的 1/4。SWA Bounded Replay 进一步把其持久 KV 缓存占用（始终位于 SSD 或主机内存中）降至约为 DeepSeek-V4-Flash 的 1/8。这些削减缓解了 HBM 与 SSD 的容量压力，同时模型整体性能显著优于 DeepSeek-V4-Flash。尽管参数量明显小于 GLM-5.3 与 Kimi-K3 等当代开源模型，DeepSeek-V4.1 在关键基准上取得了可比——在若干任务上甚至更优——的性能。

尽管相对于 DeepSeek-V4-Flash，DeepSeek-V4.1-Flash 大幅简化了若干架构组件，新引入的架构改动也带来了一些尚未完全刻画的稳健性边界。我们的内部评估覆盖了多样的测试用例与边界条件，在所评估的设置中没有观察到任何系统性的能力退化。然而，任何有限的测试集都无法覆盖所有极端输入与部署条件。CSA2 中潜在的选择错误，以及 SWA Bounded Replay 中的近似状态重建，仍可能在未经测试的边界情形下造成能力退化。接下来，我们将继续扩充压力测试与评估体系，重点关注长上下文上的稀疏检索，以及缓存恢复边界处的 SWA 状态重建。我们还将监控真实世界的工作负载，系统性地刻画潜在失效模式与稳健性边界，并进一步提升模型在极端条件下的稳健性。

随着 AI 模型展现出卓越的性能，标准评测基准已日益趋于饱和。DeepSeek-V4.1-Flash 的表现已十分接近 Fable-5 与 GPT-6 Astra 等顶级模型——在日常应用中可提供高度相近的用户体验——但在最具挑战性的任务上仍存在性能差距。虽然基准分数显示差距不大，但这种接近并不意味着该模型在复杂、高难度的推理与边缘情形上已经达到领先闭源系统的前沿能力。

因此，我们将持续更新评测协议，确保对最先进的推理边界做出严格评估。在持续降低模型成本的同时，我们认为模型智能的进一步进步将取决于数据、模型容量与强化学习的协同扩展。以 DeepSeek-V4.1-Flash 为新的起点，我们将继续探索模型能力的极限，并系统性地解决大规模数据合成与强化学习扩展中的关键挑战。我们还将积极推进模型与执行框架的协同设计（model-harness co-design），让整个联合系统共同演化、共同优化。通过同步推进降本与能力扩展，我们希望让高性能智能体更易获取、更易部署，进一步降低各行业与各场景采用 AI 技术的门槛。

参考文献

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

E. Alvarez, O. Almog, E. Chung, S. Layton, D. Stosic, R. Krashinsky, and K. Aubrey. Introducing nvfp4 for efficient and accurate low-precision inference, 2025. URL https://developer. nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-pre cision-inference/.

Anomaly. Opencode. https://github.com/anomalyco/opencode, 2026.

Anthropic. Claude code. https://code.claude.com/docs/en/overview, 2026.

Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639-3664, 2025.

Y. Bai, Q. Dong, T. Jiang, X. Lv, Z. Du, A. Zeng, J. Tang, and J. Li. Indexcache: Accelerating sparse attention via cross-layer index reuse. arXiv preprint arXiv:2603.12201, 2026.

W. Brandon, M. Mishra, A. Nrusimha, R. Panda, and J. Ragan-Kelley. Reducing transformer keyvalue cache size with cross-layer attention. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 86927-86957. Curran Associates, Inc., 2024. doi: 10.52202/07901 7-2758. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/9 e23d020c18e4c40d81c6a0fc7a46f68-Paper-Conference.pdf.

L. Chen, D. Xu, C. An, X. Wang, Y. Zhang, J. Chen, Z. Liang, F. Wei, J. Liang, Y. Xiao, et al. Powerattention: exponentially scaling of receptive fields for effective sparse attention. arXiv preprint arXiv:2503.03588, 2025.

L. Chen, W. Xie, Y. Liang, H. He, H. Zhao, Z. Yang, Z. Huang, H. Wu, H. Lu, Y. Bao, et al. Babyvision: Visual reasoning beyond language. arXiv preprint arXiv:2601.06521, 2026.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

X. Cheng, X. Yu, C. Shao, J. Li, Y. Xiong, Y. Qian, J. Zhu, S. Ma, X. Zhang, J. Ye, et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. arXiv preprint arXiv:2607.05147, 2026a.

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of

sparsity for large language models. CoRR, abs/2601.07372, 2026b. doi: 10.48550/ARXIV.2601. 07372. URL https://doi.org/10.48550/arXiv.2601.07372.

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, H. Zhang, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 4968-4990, 2026c.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066.

DataCurve. Deepswe v1.1, 2026. URL https://deepswe.datacurve.ai/.

DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL https://doi.org/10.48550/arXiv.2412.19437.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL https://doi.org/10.48550/arXiv.2405.04 434.

DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556.

DeepSeek-AI. Deepseek harness: Everything is a plugin. https://github.com/deepseek-a i/deepseek-harness, 2026a.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. CoRR, abs/2606.19348, 2026b. URL https://doi.org/10.48550/arXiv.2606.19348.

J. Dekoninck, N. Jovanovic´, I. Petrov, and M. Vechev. Matharena apex: Unconquered final-answer problems, 2025. URL https://matharena.ai/apex/.

S. Deng, Z. Ouyang, T. Pang, Z. Liu, R. Jin, S. Yu, and Y. Yang. Rmnp: Row-momentum normalized preconditioning for scalable matrix-based optimization. arXiv preprint arXiv:2603.20527, 2026.

J. Ding, S. Long, C. Pu, H. Zhou, H. Gao, X. Gao, C. He, Y. Hou, F. Hu, Z. Li, et al. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. arXiv preprint arXiv:2512.12730, 2025.

A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations, 2021. URL https://openreview.net/forum?id=YicbFdNTTy.

X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. arXiv preprint arXiv:2502.14739, 2025.

D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 23682378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246.

Y. Gao, J. Wei, Q. Zhang, Y. Cheng, S. Chen, Z. Tang, Z. Jiang, Y. Song, H. Zhang, L. Zhao, B. Yang, G. Wang, S. Cao, and F. Luo. Hysparse: A hybrid sparse attention architecture with oracle token selection and kv cache sharing. arXiv preprint arXiv:2602.03560, 2026.

S. Garre, C. Mutty, S. Mehta, and E. Chen. Chartography: A benchmark for professional chart understanding. arXiv preprint arXiv:2608.10677, 2026.

A. Glentis, J. Li, A. Han, and M. Hong. Memory-efficient llm pretraining via minimalist optimizer design. arXiv preprint arXiv:2506.16659, 2025.

Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In The Twelfth International Conference on Learning Representations, 2024.

L. Haas, G. Yona, G. D'Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. arXiv preprint arXiv:2509.07968, 2025.

D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL https://doi.org/10.48550/arXiv.2504.10356.

B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.

K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.

M. Kazemi, B. Fatemi, H. Bansal, J. Palowitch, C. Anastasiou, S. V. Mehta, L. K. Jain, V. Aglietti, D. Jindal, Y. P. Chen, et al. Big-bench extra hard. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 26473-26501, 2025.

S. Kazemzadeh, V. Ordonez, M. Matten, and T. Berg. Referitgame: Referring to objects in photographs of natural scenes. In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pages 787-798, 2014.

R. Kondra, S. Mhatre, A. Kumar, E. Chu, B. B. Ahmad, A. Nangia, R. Agarwal, A. Dasgupta, A. Sinha, B. Sridharan, K. Dave, B. Graham, G. Song, A. Rahul, W. H. Lim, A. Thangamuthu, R. Singh, D. Liu, N. Pour, C. Chen, and J. Mattern. Frontierswe v2. Proximal Blog, 2026. https://frontierswe.com/blog/v2.

H. Lee, J. Liu, D. Kim, W. Xia, Z. Zhang, C. S. Xia, and L. Zhang. Sec-bench pro: Can language models solve long-horizon software security tasks? arXiv preprint arXiv:2605.26548, 2026.

J. Li and S. Liu. Flashmla: Efficient multi-head latent attention kernels. https://github.com /deepseek-ai/FlashMLA, 2025.

J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL https://doi.org/10.48550/arXiv.2502.16982.

I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In International Conference on Learning Representations, 2019. URL https://openreview.net/forum?id=Bkg6Ri CqY7.

K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi: 10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.

J. Mao, J. Huang, A. Toshev, O. Camburu, A. L. Yuille, and K. Murphy.Generation and comprehension of unambiguous object descriptions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 11-20, 2016.

A. Marafioti, O. Zohar, M. Farré, M. Noyan, E. Bakouch, P. Cuenca, C. Zakka, L. B. Allal, A. Lozhkov, N. Tazi, V. Srivastav, J. Lochner, H. Larcher, M. Morlon, L. Tunstall, L. von Werra, and T. Wolf. Smolvlm: Redefining small and efficient multimodal models. arXiv preprint arXiv:2504.05299, 2025.

R. Marten, A. Shaw, I. Bercovich, B. Droste, T. Cerruti, S. Dillmann, R. Wang, D. Wahdany, A. Hart, K. Krauth, ScaleAI, Snorkel AI, Turing, gNucleus AI, Boolean AI, N. Carlini, S. Lyu, A. Wei, A. Khatua, B. Plüster, C. Dwivedi, C. Sutcliffe, Yuming, D. Tivris, D. Wang, H. W. Goh, H. Xing, H. Lin, I. Salia, J. Seol, J. Bao, J. Ouyang, J. Park, L. Walsh, L. Kong, M. Ivanov, M. Ubl, M. Liamets, O. Menis, P. Migdal, Q. Bao, R. Movva, R. Ben Chaim, N. Srinath, S. Bogdanik, S. Yadav, S. Benjamin, T. Kung, W. Hughes, X. Lan, H. Gupta, S. Mishra, C. Wang, H. He, J. Tu, K. Montgomery, Z. Tu, A. Naik, D. Mortensen, I. Zhang, Y. Mathur, E. Liu, K. Singh, M. Yu, S. Feng, V. Gangal, Z. Tao, S. Ruan, J. Mueller, J. Cabezas, J. Bauer, K. X. Li, R. Zhang, A. Feller, A. Madayan, L. Chen, B. Feuer, X. Li, B. Li, H. Raj, S. Galler, L. Shi, I. Segal, K. Buchanan, S. P., R. Desai, A. Schneider, C. Settles, X. Lin, M. Nezhurina, A. Wang, M. Kowalczyk, J.-X. Zhao, S. Satia, J. Hu, S. Atef, K. Chen, S. Vance, G. Segato, J. Jitsev, A. Dimakis, M. Merrill, A. Konwinski, and L. Schmidt. Terminal-Bench, Aug. 2026a. URL https://github.com/harbor-framework/terminal-bench.

R. Marten, A. Shaw, A. Konwinski, and Terminal-Bench Contributors. Terminal-bench 3.0: Harder tasks for better agents. https://www.tbench.ai/news/terminal-bench-3-0, 2026b. An Open Benchmark for Agent Work in Terminal Environments.

M. Mathew, D. Karatzas, and C. Jawahar. Docvqa: A dataset for vqa on document images. In 2021 IEEE Winter Conference on Applications of Computer Vision (WACV), pages 2199-2208. IEEE, 2021.

M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.

V. K. Nagaraja, V. I. Morariu, and L. S. Davis. Modeling context between objects for referring expression understanding. In European conference on computer vision, pages 792-807. Springer, 2016.

Y. Nesterov. A method of solving a convex programming problem with convergence rate O(1/k). Soviet Mathematics Doklady, 27:372-376, 1983.

OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/ARXIV.2508.10925. URL https://doi.org/10.48550/arXiv.2508.10925.

OpenAI. Codex. https://github.com/openai/codex, 2026.

L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity's last exam. arXiv preprint arXiv:2501.14249, 2025.

Y. Qian, S. Liu, and Y. Li. Deepselect: High-performance topk kernels for deepseek sparse attention and sampling. https://github.com/deepseek-ai/DeepSelect, 2026.

D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

J. Roberts, M. R. Taesiri, A. Sharma, A. Gupta, S. Roberts, I. Croitoru, S.-V. Bogolin, J. Tang, F. Langer, V. Raina, et al. Zerobench: An impossible visual benchmark for contemporary large multimodal models. arXiv preprint arXiv:2502.09696, 2025.

B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.

M. Scetbon, C. Ma, W. Gong, and E. Meeds. Gradient multi-normalization for efficient LLM training. In The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025. URL https://openreview.net/forum?id=oanhUGY6un.

N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

N. Shazeer and M. Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In International conference on machine learning, pages 4596-4604. PMLR, 2018.

D. Shepard and R. Salimans. Automationbench. arXiv preprint arXiv:2604.18934, 2026.

F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?i d=fR3wGCk-IXp.

Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, Q. Zhang, J. Wang, and F. Wei. You only cache once: Decoder-decoder architectures for language models. Advances in Neural Information Processing Systems, 37:7339-7361, 2024.

Y. Sun, X. Han, W. Zhang, Y. Pang, T. Wang, Y. Cao, Y. Huang, C. Duroiu, H. Zhang, J. Lin, et al. Agents' last exam. arXiv preprint arXiv:2606.05405, 2026a.

Y. Sun, Y. Zhang, L. Dong, J. Wang, and F. Wei. You only index once: Cross-layer sparse attention with shared routing. arXiv preprint arXiv:2606.06467, 2026b.

M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

K. Team, T. Bai, Y. Bai, Y. Bao, J. Cai, X. Cai, P. Cao, Y. Cao, Z. Chai, Y. Charles, et al. Kimi k3: Open frontier intelligence. arXiv preprint arXiv:2607.24653, 2026a.

K. Team, T. Bai, Y. Bai, Y. Bao, S. Cai, Y. Cao, Z. Chai, Y. Charles, H. Che, C. Chen, et al. Kimi k2.5: Visual agentic intelligence. arXiv preprint arXiv:2602.02276, 2026b.

M. L. Team, B. Wang, B. Xiao, B. Zhang, B. Rong, B. Chen, C. Wan, C. Zhang, C. Huang, C. Chen, et al. Longcat-flash-omni technical report. arXiv preprint arXiv:2511.00279, 2025.

S. Tong, E. Brown, P. Wu, S. Woo, M. Middepogu, S. C. Akula, J. Yang, S. Yang, A. Iyer, X. Pan, A. Wang, R. Fergus, Y. LeCun, and S. Xie. Cambrian-1: A fully open, vision-centric exploration of multimodal llms, 2024.

L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL https://doi.org/10.48550/arX iv.2408.15664.

X. Wang, C. Xu, H. Cao, R. Tian, W. Zhao, K. Yu, and C. Zhao. Tilekernels. https://github.c om/deepseek-ai/TileKernels, 2026a.

Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL https://doi.org/10.48550/arXiv.2406.01574.

Z. Wang, N. Schiller, H. Li, S. S. Narayana, M. Nasr, N. Carlini, X. Qi, E. Wallace, E. Bursztein, L. Invernizzi, et al. Exploitgym: Can ai agents turn security vulnerabilities into real attacks? arXiv preprint arXiv:2605.11086, 2026b.

Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents' real-world cybersecurity capabilities at scale. In International Conference on Learning Representations, volume 2026, pages 123341-123386, 2026c.

Z. Wen, Y. Shi, J. Wang, P. Luo, L. Qiao, D. Li, and T. Sun. Sron: State-free llm training via row-wise gradient normalization. 2025.

Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifoldconstrained hyper-connections, 2026. URL https://arxiv.org/abs/2512.24880.

R. Xu, J. Li, and Y. Lu. On the width scaling of neural optimizers under matrix operator norms i: Row/column normalization and hyperparameter transfer. arXiv preprint arXiv:2603.09952, 2026a.

Y. Xu, F. Meng, F. Jiang, Y. Wang, R. Zhou, Z. Wang, J. Wu, Z. Pan, X. Tang, W. Pei, et al. Hisa: Efficient hierarchical indexing for fine-grained sparse attention. arXiv preprint arXiv:2603.28458, 2026b.

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024. URL https://arxi v.org/abs/2405.15793.

J. Yang, K. Lieret, J. Ma, P. Thakkar, D. Pedchenko, S. Sootla, E. McMilin, P. Yin, R. Hou, G. Synnaeve, D. Yang, and O. Press. Programbench: Can language models rebuild programs from scratch?, 2026. URL https://arxiv.org/abs/2605.03546.

L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg. Modeling context in referring expressions. In European conference on computer vision, pages 69-85. Springer, 2016.

J. Yuan, J. Zou, S. Wang, Y. Liu, and F. Nie. Nora: Normalized orthogonal row alignment for scalable matrix optimizer. arXiv preprint arXiv:2605.03769, 2026.

X. Yue, T. Zheng, Y. Ni, Y. Wang, K. Zhang, S. Tong, Y. Sun, B. Yu, G. Zhang, H. Sun, et al. Mmmupro: A more robust multi-discipline multimodal understanding benchmark. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15134-15186, 2025.

M. Zechner. Pi: The coding-agent harness you can make your own. https://github.com/e arendil-works/pi, 2026.

R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791-4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472.

A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie, et al. Glm-5: from vibe coding to agentic engineering. arXiv preprint arXiv:2602.15763, 2026.

X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In 2023 IEEE/CVF International Conference on Computer Vision (ICCV), pages 11941-11952. IEEE, 2023.

B. Zhang and R. Sennrich.Root mean square layer normalization.Advances in neural information processing systems, 32, 2019.

Y. Zhang. On the principles behind neural network optimizers. arXiv preprint arXiv:2608.16760, 2026.

Y. Zhang, C. Chen, T. Ding, Z. Li, R. Sun, and Z.-Q. Luo. Why transformers need adam: A hessian perspective. Advances in neural information processing systems, 37:131786-131823, 2024.

Y. Zhang, C. Chen, Z. Li, T. Ding, C. Wu, D. D. Kingma, Y. Ye, Z.-Q. Luo, and R. Sun. Adammini: Use fewer learning rates to gain more.In International Conference on Learning Representations, volume 2025, pages 28033-28063, 2025a.

Z. Zhang, Y. Zhong, Y. Jiang, H. Hu, J. Sun, Z. Ge, Y. Zhu, D. Jiang, and X. Jin. Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models. In Proceedings of the ACM SIGCOMM 2025 Conference, pages 24-38, 2025b.

C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. https://github.com/deepseek-ai/DeepGEMM, 2025.

W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364.

T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL http s://openreview.net/forum?id=YrycTjllL0.

作者名单

按名字首字母排序。带 * 者表示已离开团队的成员。

Research & Engineering: Anyi Xu, B. Li, Bangcai Lin, Bing Xue, BingCheng Xian, Bingzheng Xu, Bochao Wu, Bowei Zhang, Boyi Deng, C.C. Yu, Chao Jin, Chaofan Lin, Chen Dong, Chenbing Wang, Chenfan Feng, Chengda Lu*, Chenggang Zhao, Chengqi Deng, Chengyuan Zhang, Chenhao Xu, Chenqi Zhao, Chenze Shao, Chuhao Wang, Chuqi Zhang, Damai Dai, Dejian Yang, Deli Chen, Di Huang, Di Wu, Donghao Li, Erhang Li, Eric Fu, F. Zhou, Fangwei Zhou, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanglin Li, Guanting Chen*, Guoai Cao, Guofan Fan, Guolai Meng, Guowei Li, Haichuan Zhang, Haiyang Ma, Haiyang Shen, Han Li, Han Yu, Han Zhang, Hangyuan Deng, Hanwei Xu, Hanxiang Xu, Hanxun Zhong, Hao Guo, Hao Jiang, Hao Li, Hao Qin, Haodong Wen, Haofen Liang, Haofeng Huang, Haohua Liu, Haoling Zhang, Haoming Luo, Haoran Yang, Haotian Xu*, Haotian Yuan, Haoting Huang, Haowen Luo, Haoyang Cai, Haoyu Chen, Haozhe Ji, Hengran Zhang, Hengrui Wang, Hengxu Wu, Honghui Ding, Hongxuan Tang, Huadong Wang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J. Yang, J.H. Jin, J.H. Zhang, J.X. Zou, Jia Yu, Jiahui Zhou, Jiajun Chen, Jialiang Huang, Jialin Zhao, Jiamin Tang, Jian Zhou, Jianan Tong, Jianwen Li, Jiaqi Zhu, Jiarui Wang, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiaying Ding, Jibai Lu, Jiewen Hu, Jin Yan, Jincheng Zhai, Jingchang Chen, Jingcheng Hu, Jingli Zhou, Jingsheng Xu, Jingting Xiang, Jingyan Yun, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jinpeng Wang, Jinyi Chen, Jinyi Hu, Jiping Yu, Jueliang Guo, Junbo Pei, Junbo Sun, Junguang Jiang, Junjie Qiu, Junkang Zhou, Junqi Liu, Junren Li, Junxian Li, Junxiao Song, Junyi Guo, Kai Dong, Kaifeng Chen, Kaige Gao, Kang Guan, Kangdong Yuan, Ke Hong, Ke Xu, Kefan Zhao, Kexin Ji, Kexin Zhang, Kexing Zhou, Kuai Yu, Lan Zhang, Lean Wang, Lecong Zhang, Lei Wang, Letian Gao, Liang Zhao, Liansheng Xu, Lihua Guo, Lingxiao Luo, Lingyue Fu, Litao Deng, Litong Wang, Liyue Zhang, Longhao Chen, Lu Chen, Luotian Huang, Luyao Ma, Luyao Wang, M.S. Di, Max Mei, Menghao Ye, Miao Cui, Mingchuan Zhang, Minghua Zhang*, Minghui Tang, Mingjing Zhang, Mingqi Wei, Mingshu Chen, Mingxing Liu, Mingxu Zhou, Mingyu Xu, Mingyu Yang, Mingze Wang, Muyang Chen, Ni Shentu, Ning Wang, Niufang Ning, Panpan Huang, Peixin Cong, Peiyi Wang, Peiyuan Xin, Pengfei Ren, Pengfei Yan, Pengle Zhang, Qi Kang, Qi Tang, Qiancheng Wang, Qiang Li, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qizhou Guo, Rongxian Xu, Rui Ding, Rui Hu, Rui Tian, Rui Yu, Ruidong Zhu, Ruifan Xu, Ruihan Yang, Ruihang Xia, Ruijie Lu, Ruilin Geng, Ruipeng Hong, Ruiqi Ge, Ruisong Zhang, Ruize Sun, Ruizhe Pan, Runji Wang, Runqian Chen, Runxin Xu, Ruohong Tian, Ruomeng Shen, Ruoyu Zhang, Ryan X., S.H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang

Chen, Shaofei Cai, Shaoheng Nie, Shaoyuan Chen, Shengding Hu, Shengkai Lin, Shengwen Ran, Shengyu Liu, Shengyuan Jia, Shi Bai, Shi Feng, Shicheng Xu, Shichun Liu, Shiqiang Hu*, Shirong Ma, Shiyu Wang, Shiyuan Feng, Shufan Gong, Shuhan Lin, Shuiping Yu, Shunfeng Zhou, Shuo Yang, Shuomeng Wang, Shuting Guo, Shuting Pan, Shuying Yu, Sinuo Cao, Siyi Lin, Sizhe Chen, Songyang Chen, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji*, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tongrui Xiong, Wangding Zeng, Wei Liu, Wei Zhang, Weibin Xu, Weihao Zeng, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao*, Wenjun Gao, Wenkai Shao, Wenkai Yang, Wenli Zhang, Wenlu Wang, Wenlve Huang, Wenqian Yan, Wentao Zhang, Xi Gao, Xiang He, Xiang Li, Xiangli Li, Xiangwen Wang, Xiangying Zhang, Xiankui Wei, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojian Qu, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaoyao Zou, Xiaoyuan Li, Xicheng Guo, Xieting Chu,

附录 A 评测细节

A.1 脚手架配置

所有脚手架（scaffold）均在 Linux 任务容器中运行，并采用表 4 所列的共享评测设置。每次运行都以基准任务描述作为起点，使用该脚手架的运行时或评测集成所提供的提示词、任务模板与工具定义。我们不添加任何实验性的系统提示词。

表 5 Max 推理强度下不同 Claude Code 版本的性能

基准（指标）	v2.1.105	v2.1.238	v2.1.251	v2.1.259	平均
DeepSWE v1.1（Resolved）	68.4	68.7	69.8	68.6	68.9
Terminal-Bench v2.1（Pass@1）	87.3	88.4	88.0	87.6	87.8

注：平均分由四个版本未舍入的 Pass@1 分数计算得到。

- Claude Code（v2.1.105、v2.1.238、v2.1.251、v2.1.259）。我们使用 Claude Agent SDK 及各版本原生的工具接口。表 4 报告的是 v2.1.251 的结果，表 5 则比较全部四个版本。
- Codex（v0.147.0）。我们使用标准的 app-server 模式，并对工具 schema 作了适配。
- OpenCode（v1.18.15）。我们使用其 build agent，配备 shell 工具与文件工具，以及原生的任务委派功能。
- Pi（v0.84.2）。我们使用 RPC 模式，配备文件与 shell 工具，外加一个暴露 `search`、`open_page` 与 `find_in_page` 的搜索扩展。
- mini-SWE。我们使用 `mini_swe_v2` 移植版，仅提供单条 bash 工具；每一轮都必须发起一次工具调用，并以一个提交标记收尾。
- DeepSeek Harness（DSH）。Minimal 配置使用单条 bash 工具；Standard 配置使用完整的 `sdk` profile 和 26 个初始函数工具（含网络搜索与网页抓取）；PTC 配置使用 `run_code` 运行 TypeScript 程序，底层调用 24 个工具。Standard 与 PTC 使用 v0.1.1+custom.202609011522 版本。

mini-swe-agent，commit 04d809ceab9d。

A.2 不同脚手架下的推理强度

在图 11 的全部六块面板中，提高推理强度都会拉长轨迹：在每一种脚手架与基准的组合下，每条轨迹的平均输出词元（token）数都随强度单调增长。Pass@1 只是宽松地跟随这一增长：总体上有所提升，但并非单调，大多数面板在中档强度处出现平台期和回落。三种脚手架的标定方式也各不相同：在 DeepSWE v1.1 上，Claude Code 的曲线最平缓，额外付出的词元较少；DeepSeek Harness（Minimal）起点最低，却以最大的词元代价换取最多的提升，mini-SWE 介于两者之间。在 Terminal-Bench v2.1 上，三者的曲线被压缩到一个狭窄区间内，并且在最大强度处的排序以 DeepSeek Harness（Minimal）为首，mini-SWE 和 Claude Code 随后：当任务已经接近饱和时，脚手架的选择至少与强度档位同样重要。

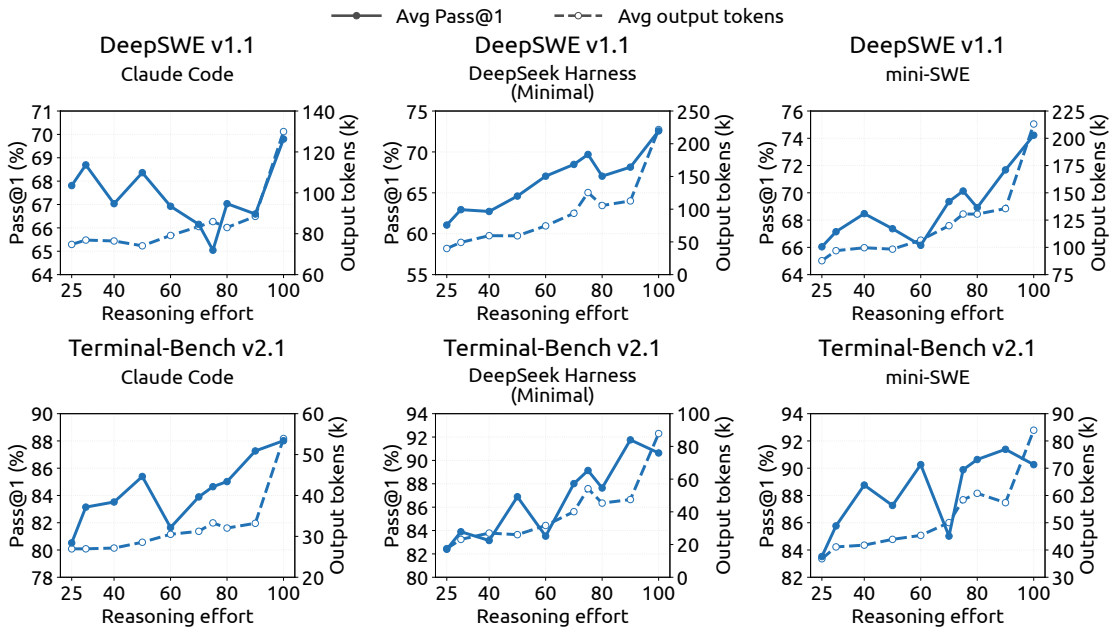


图 11　推理强度稳定地驱动轨迹长度，但在各个编码脚手架之间与准确率仅呈弱相关。每个面板绘制 Pass@1（%，实线，左轴）与每条轨迹的平均输出词元数（k，虚线，右轴）随推理强度设置的变化，对象为 DeepSWE v1.1（上排）与 Terminal-Bench v2.1（下排）下的三种智能体脚手架：Claude Code、DeepSeek Harness（Minimal）与 mini-SWE。所有面板均来自同一检查点。

A.3 不同推理强度下推理基准的详细结果

图 12 表明，推理强度设置对输出长度和准确率都提供了平滑且表现良好的控制，并且在覆盖竞赛数学、科学问答、开放域知识与代码的全部八个基准上保持一致。在长度一侧，把强度从 25 提高到 100，每个基准的平均响应长度都会可预测地随之放大：增幅一致地落在 2.0–3.1 倍之间，AIME 2026 上从每条响应 4.6k 词元增至 11.4k 词元，MathArena Apex 2025 上从 29.1k 增至 86.1k 词元，且没有失控增长或异常现象，因此任何档位的计算成本都可以预先估算。准确率沿同样平滑的轨迹变化，在每个基准上都朝正确的方向响应，没有任何基准随强度提高而出现退化：MathArena Apex 2025 提升 +40.3 分（25.3% 提升至 65.6%），Apex 2025 Shortlist 提升 +11.5，即便是早已饱和的基准也保持稳定（GPQA Diamond +1.3，LiveCodeBench +2.6），而 AIME 2026 则达到完整的 100%。模型由此提供了一个单一且可靠的调节旋钮，以受控、可预测的方式移动成本—准确率工作点，使每套部署都能选择与其时延和计算预算相匹配的强度档位，而无须牺牲准确率。

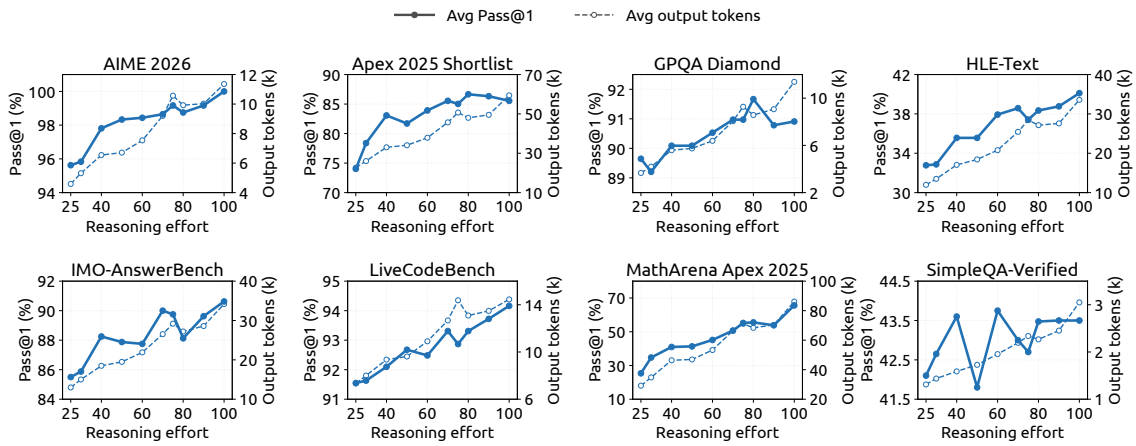


图 12　八个推理密集型基准上性能与输出长度随推理强度的变化关系。每个面板绘制 Pass@1（实线，左轴）与每条响应的平均输出词元数（虚线，右轴），推理强度取值从 25 变化到 100。

附录 B 推理强度控制中的指数词元惩罚

标量形式的强度变量在不施加硬性词元预算的前提下，为部署阶段提供了一种对成本—质量权衡的控制手段。在强化学习期间，较低的强度档位施加更强的词元惩罚，较高的强度档位则允许更多计算。本节为这一指数惩罚调度给出一个简化的动机说明。

对于在强度档位 b 下生成了 ℓ 个推理词元的轨迹，其长度扣减为

$$r^{\text{len}}(\ell, b) = -\min\left\{C_{\max},\ k(b)\frac{\ell}{L_{\text{norm}}}\right\}, \tag{11}$$

其中 L_{norm} 为参考长度，$C_{\max}$ 限制了扣减上限。随强度变化的词元惩罚系数按指数规律随强度衰减：

$$k(b) = k_0 \exp\left(-\frac{b - b_{\min}}{\tau}\right), \tag{12}$$

其中 k_0 为最低强度档位 $b_{\min}$ 处的惩罚系数，τ 控制惩罚的衰减速率。

为说明这一选择的动机，考虑一个固定的问题 x。令 $p_x(\ell)$ 表示在产生 ℓ 个推理词元后该问题被解出的概率。在未触发的扣减上限区域内，将偏好的推理长度定义为

$$\ell_x^*(b) \in \arg\max_{\ell \geq 0}\left[p_x(\ell) - k(b)\frac{\ell}{L_{\text{norm}}}\right]. \tag{13}$$

对于内点最优解，一阶条件为

$$p_x'(\ell_x^*(b)) = \frac{k(b)}{L_{\text{norm}}}, \tag{14}$$

此时，再多一个词元推理的边际收益与其边际长度惩罚在边际上恰好相抵。

我们假设，这一边际收益在相关的长度范围内近似按指数衰减：

$$p_x'(\ell) \approx a_x \exp\left(-\frac{\ell}{s_x}\right), \tag{15}$$

其中 $a_x > 0$ 为随问题实例而异的尺度参数，$s_x > 0$ 决定衰减速率。将式 (15) 代入式 (14) 并求解，可得

$$\ell_x^*(b) \approx C_x - s_x \log k_0 + \frac{s_x}{\tau}(b - b_{\min}), \tag{16}$$

其中 $C_x = s_x \log(a_x L_{\text{norm}})$ 与 b 无关。因此，在这一局部模型下，指数惩罚调度使所设置的强度与偏好的推理长度之间呈现一个简单的一阶仿射趋势。

对于两个强度档位 $b_2 > b_1$，相应的预测长度差为

$$\ell_x^*(b_2) - \ell_x^*(b_1) \approx \frac{s_x}{\tau}(b_2 - b_1). \tag{17}$$

由此可见，k_0 主要控制朝向更短推理的整体压力，而 τ 控制预测结果对强度的敏感程度。

这一推导是奖励层面的局部近似，并不主张实测的平均长度必须呈线性或逐点单调。实际表现可能偏离该模型，原因在于：推理强度指令会直接改变推理策略；生成过程本身是随机的；智能体轨迹包含的轮数各不相同；而且子组的奖励归一化会改变优化强度。此外，该分析假设存在一个惩罚上限未被触发的内点解。一旦触及上限，边际词元惩罚变为零，此时必须单独考虑封顶区域。